Senior Backend Engineer (Golang)

Complete Interview Preparation Guide

Production-Grade Depth • 14 Technologies • Full Cheat Sheet

1. GOLANG — DEEP DIVE

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

Go is a statically-typed, compiled language by Google (2009). It solves the C++ compilation slowness and Java verbosity while providing built-in concurrency primitives. Use it for high-throughput microservices, CLIs, and network systems where you need predictable latency without GC pauses of JVM magnitude.

1.1 Go Runtime, Compilation Model, GOPATH vs Go Modules

Go compiles to a single statically-linked binary (no JVM, no interpreter). The runtime handles goroutine scheduling, GC, and stack management.

GOPATH (Legacy) vs Go Modules (Modern)

- GOPATH: all code lives under $GOPATH/src — tight coupling, no versioning
- Go Modules (1.11+): go.mod declares module path and dependency versions — use this always

```
# Initialize a new module
go mod init github.com/yourcompany/myservice

# go.mod example
module github.com/yourcompany/myservice

go 1.22

require (
    github.com/jackc/pgx/v5 v5.5.5
    google.golang.org/grpc v1.63.2
)

# Download dependencies
go mod tidy          # add missing, remove unused
go mod download      # download all deps to module cache

# Workspace mode (multiple modules)
go work init ./service-a ./service-b
```

Compilation Model

- go build: compiles to binary; uses build cache (~/.cache/go-build)
- go run: compiles + executes in temp dir (NOT for production)
- Cross-compilation: GOOS=linux GOARCH=amd64 go build

```
# Production build with version injection
CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build \
  -ldflags="-X main.Version=v1.2.3 -X main.BuildTime=$(date -u +%Y%m%dT%H%M%S) -w -s" \
  -o ./bin/server ./cmd/server

# -w: strip debug info; -s: strip symbol table (reduces binary ~30%)
```

1.2 Goroutines: Lifecycle, G-M-P Scheduling, Goroutine Leaks

Goroutines are lightweight user-space threads (2KB initial stack, grows dynamically). The runtime scheduler multiplexes them on OS threads using the G-M-P model.

G-M-P Model

- G (Goroutine): the unit of execution; holds stack, PC, state
- M (Machine): OS thread; actually runs Go code on CPU
- P (Processor): logical CPU context; holds run queue (max 256 goroutines); GOMAXPROCS controls P count

When a goroutine blocks on IO, the M detaches from P, P finds another M (or creates one), keeping CPUs busy.

```go
// Goroutine lifecycle
func main() {
    done := make(chan struct{})

    go func() {
        defer close(done)            // signal when done
        defer func() {               // always recover panics in goroutines
            if r := recover(); r != nil {
                log.Printf("recovered: %v", r)
            }
        }()
        doWork()
    }()

    select {
    case <-done:
        fmt.Println("completed")
    case <-time.After(5 * time.Second):
        fmt.Println("timeout")
    }
}

// GOMAXPROCS — set to number of CPUs
runtime.GOMAXPROCS(runtime.NumCPU())
```

Goroutine Leak Detection

```go
// LEAK: goroutine blocked forever waiting on channel nobody writes to
func leak() {
    ch := make(chan int)
    go func() {
        val := <-ch  // blocks forever if nobody sends
        fmt.Println(val)
    }()
    // ch goes out of scope; goroutine leaks
}

// FIX: use context for cancellation
func noLeak(ctx context.Context) {
    ch := make(chan int, 1)
    go func() {
        select {
        case val := <-ch:
            fmt.Println(val)
        case <-ctx.Done():
            return  // goroutine exits cleanly
```

```go
        }
    }()
}

// Detect leaks with goleak in tests
import "go.uber.org/goleak"

func TestMyFunc(t *testing.T) {
    defer goleak.VerifyNone(t)
    // ... test code
}
```

> **Gotcha:** runtime.NumGoroutine() in metrics — a monotonically increasing count means goroutine leak

1.3 Channels: Buffered vs Unbuffered, Directional, Select

```go
// Unbuffered: sender blocks until receiver ready (synchronous rendezvous)
ch := make(chan int)

// Buffered: sender blocks only when buffer full
ch := make(chan int, 100)

// Directional channels (enforce at compile time)
func producer(out chan<- int) { out <- 42 }
func consumer(in <-chan int)  { val := <-in; _ = val }

// Select: multiplex over channels
func process(ctx context.Context, jobs <-chan Job, results chan<- Result) {
    for {
        select {
        case job, ok := <-jobs:
            if !ok { return }  // channel closed
            results <- process(job)
        case <-ctx.Done():
            return
        }
    }
}

// Fan-out pattern
func fanOut(in <-chan int, workers int) []<-chan int {
    channels := make([]<-chan int, workers)
    for i := 0; i < workers; i++ {
        ch := make(chan int, 10)
        channels[i] = ch
        go func(out chan<- int) {
            defer close(out)
            for v := range in { out <- v * 2 }
        }(ch)
    }
    return channels
}

// Common pattern: pipeline
func generator(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, n := range nums { out <- n }
    }()
    return out
}
```

```go
func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in { out <- n * n }
    }()
    return out
}
```

> **GOTCHA:** Sending to a nil channel blocks forever. Closing a nil channel panics. Closing an already-closed channel panics. Always use the ok idiom: val, ok := <-ch

1.4 sync Package: Mutex, RWMutex, WaitGroup, Once, Atomic

```go
// Mutex — mutual exclusion
type SafeCounter struct {
    mu    sync.Mutex
    count int
}
func (c *SafeCounter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()  // ALWAYS defer unlock
    c.count++
}

// RWMutex — multiple readers OR one writer
type Cache struct {
    mu    sync.RWMutex
    items map[string]string
}
func (c *Cache) Get(key string) (string, bool) {
    c.mu.RLock()    // multiple goroutines can hold RLock simultaneously
    defer c.mu.RUnlock()
    v, ok := c.items[key]
    return v, ok
}
func (c *Cache) Set(key, val string) {
    c.mu.Lock()     // exclusive write lock
    defer c.mu.Unlock()
    c.items[key] = val
}

// WaitGroup — wait for goroutines to finish
func processAll(items []Item) {
    var wg sync.WaitGroup
    for _, item := range items {
        wg.Add(1)                    // Add BEFORE launching goroutine
        go func(it Item) {
            defer wg.Done()          // Done in defer
            process(it)
        }(item)                      // pass item as arg (avoid closure capture bug)
    }
    wg.Wait()
}

// sync.Once — execute exactly once (init, singleton)
var (
    instance *DB
    once     sync.Once
)
func GetDB() *DB {
    once.Do(func() { instance = connectDB() })
```

```
    return instance
}

// Atomic operations (no mutex needed for simple counters)
var counter int64
atomic.AddInt64(&counter, 1)
val := atomic.LoadInt64(&counter)
atomic.StoreInt64(&counter, 0)
atomic.CompareAndSwapInt64(&counter, old, new)
```

> **GOTCHA:** sync.Mutex must NOT be copied after first use. If you embed a Mutex in a struct, pass the struct by pointer, never by value.

1.5 Context Package: Cancellation, Deadlines, Timeouts, Propagation

```
// Context hierarchy — cancellation propagates down
func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()  // ALWAYS call cancel to release resources

    if err := doRequest(ctx); err != nil {
        log.Fatal(err)
    }
}

// Passing context through call stack
func doRequest(ctx context.Context) error {
    req, err := http.NewRequestWithContext(ctx, "GET", url, nil)
    if err != nil { return err }
    resp, err := http.DefaultClient.Do(req)
    // ...
}

// Context values — only for request-scoped data (not configuration)
type contextKey string
const requestIDKey contextKey = "requestID"

func withRequestID(ctx context.Context, id string) context.Context {
    return context.WithValue(ctx, requestIDKey, id)
}
func getRequestID(ctx context.Context) string {
    id, _ := ctx.Value(requestIDKey).(string)
    return id
}

// Check context in loops
func longOp(ctx context.Context) error {
    for i := 0; i < 1000; i++ {
        select {
        case <-ctx.Done():
            return ctx.Err()  // context.Canceled or context.DeadlineExceeded
        default:
        }
        doIteration(i)
    }
    return nil
}

// context.WithDeadline vs WithTimeout
deadline := time.Now().Add(5 * time.Second)
ctx1, cancel1 := context.WithDeadline(context.Background(), deadline)
ctx2, cancel2 := context.WithTimeout(context.Background(), 5*time.Second) // equivalent
```

> **GOTCHA:** Never store context in a struct field — always pass as first function argument. Never use context.Background() deep in business logic; always accept ctx as parameter.

1.6 Error Handling: Sentinel Errors, Wrapping, errors.Is/As, Custom Types

```go
// Sentinel errors — predefined errors for comparison
var (
    ErrNotFound   = errors.New("not found")
    ErrPermission = errors.New("permission denied")
)

// Error wrapping with %w (Go 1.13+)
func getUser(id int) (*User, error) {
    u, err := db.Find(id)
    if err != nil {
        return nil, fmt.Errorf("getUser id=%d: %w", id, err)
    }
    return u, nil
}

// errors.Is: check error identity (traverses wrap chain)
if errors.Is(err, ErrNotFound) {
    http.Error(w, "not found", 404)
}

// errors.As: unwrap to specific type
var pgErr *pgconn.PgError
if errors.As(err, &pgErr) {
    log.Printf("postgres code: %s", pgErr.Code)
}

// Custom error types
type ValidationError struct {
    Field   string
    Message string
}
func (e *ValidationError) Error() string {
    return fmt.Sprintf("validation error: field=%s msg=%s", e.Field, e.Message)
}

// Production error handling pattern
type AppError struct {
    Code    int
    Message string
    Err     error
}
func (e *AppError) Error() string { return fmt.Sprintf("[%d] %s: %v", e.Code, e.Message,
e.Err) }
func (e *AppError) Unwrap() error  { return e.Err }  // enable errors.Is/As traversal

func (s *Service) CreateOrder(ctx context.Context, req CreateOrderReq) error {
    if err := validate(req); err != nil {
        return &AppError{Code: 400, Message: "invalid request", Err: err}
    }
    if err := s.repo.Save(ctx, req); err != nil {
        return &AppError{Code: 500, Message: "failed to save order", Err: err}
    }
    return nil
}
```

1.7 Interfaces: Implicit Implementation, Empty Interface, Assertions

```
// Implicit implementation — no 'implements' keyword
type Writer interface {
    Write(p []byte) (n int, err error)
}

type FileWriter struct { f *os.File }
func (fw *FileWriter) Write(p []byte) (n int, err error) { return fw.f.Write(p) }
// FileWriter automatically implements Writer

// Interface best practices
// GOOD: small, focused interfaces (ISP)
type Reader interface { Read(p []byte) (n int, err error) }
type Closer interface { Close() error }
type ReadCloser interface { Reader; Closer }  // composition

// Empty interface (any) — use sparingly
func log(v any) { fmt.Println(v) }

// Type assertions
var i any = "hello"
s, ok := i.(string)  // safe assertion
if !ok { log.Fatal("not a string") }

// Type switch
func describe(i any) {
    switch v := i.(type) {
    case int:    fmt.Printf("int: %d
", v)
    case string: fmt.Printf("string: %s
", v)
    case []byte: fmt.Printf("bytes: %d
", len(v))
    default:     fmt.Printf("unknown: %T
", v)
    }
}

// Interface segregation in service layer
type UserRepository interface {
    FindByID(ctx context.Context, id int64) (*User, error)
    Save(ctx context.Context, u *User) error
}
type Service struct { repo UserRepository }  // depends on interface, not concrete type
```

1.8 Memory Model: Heap vs Stack, Escape Analysis, GC Pressure

```
// Stack: local variables that don't escape function
// Heap: allocations that outlive the function / shared

// Escape analysis — run this to see:
go build -gcflags="-m -m" ./...

// Example: what escapes to heap?
func stackAlloc() int {
    x := 42          // stays on stack
    return x
}

func heapAlloc() *int {
    x := 42
```

```go
    return &x         // x escapes to heap (returned pointer)
}

// Reduce GC pressure patterns:
// 1. sync.Pool for frequently allocated objects
var bufPool = sync.Pool{
    New: func() any { return make([]byte, 0, 4096) },
}
func handler(w http.ResponseWriter, r *http.Request) {
    buf := bufPool.Get().([]byte)
    defer func() {
        buf = buf[:0]
        bufPool.Put(buf)
    }()
    // use buf
}

// 2. Pre-allocate slices when size known
items := make([]Item, 0, expectedSize)

// 3. Avoid string->[]byte conversions in hot paths
// BAD:  string(byteSlice) in loop
// GOOD: keep as []byte or use strings.Builder

// GC tuning
// GOGC=100 (default): GC when heap doubles since last GC
// GOGC=200: GC less often, higher memory usage
// GOMEMLIMIT (Go 1.19+): hard memory limit
os.Setenv("GOMEMLIMIT", "512MiB")  // use in container envs
```

1.9 Pointers, Slice Internals, Map Internals

```go
// Slice header: {pointer, len, cap}
s := make([]int, 3, 10)  // len=3, cap=10
s2 := s[1:3]             // shares backing array! {ptr+1, 2, 9}

// Modifying s2 modifies s
s2[0] = 99
fmt.Println(s[1])  // 99 — same backing array

// Append beyond cap creates NEW backing array
s3 := append(s2, 1, 2, 3, 4, 5, 6, 7, 8)  // new array allocated
s3[0] = 100
fmt.Println(s2[0])  // still 99 — different arrays now

// GOTCHA: slice of slice memory leak
func getFirst100(data []byte) []byte {
    return data[:100]  // BAD: data[101:] still referenced, won't be GC'd
}
func getFirst100Fixed(data []byte) []byte {
    result := make([]byte, 100)
    copy(result, data[:100])  // GOOD: independent copy
    return result
}

// Map internals: hash map with open addressing
// Maps are NOT safe for concurrent use!
m := make(map[string]int, 100)  // hint initial size

// Concurrent map
var mu sync.RWMutex
mu.RLock()
v := m["key"]
```

```
mu.RUnlock()

// OR use sync.Map (for high-read/low-write with many goroutines)
var sm sync.Map
sm.Store("key", 42)
val, ok := sm.Load("key")
```

1.10 Defer, Panic, Recover — Real Patterns and Misuses

```
// Defer executes LIFO when surrounding function returns
func readFile(path string) (err error) {
    f, err := os.Open(path)
    if err != nil { return }
    defer f.Close()  // guaranteed cleanup

    // Named return + defer for error enrichment
    defer func() {
        if err != nil {
            err = fmt.Errorf("readFile %s: %w", path, err)
        }
    }()

    // ... read file
    return
}

// Defer in loop — WRONG
for _, f := range files {
    defer f.Close()  // all defers run at function exit, not loop iteration!
}
// CORRECT
for _, f := range files {
    func() {
        defer f.Close()
        process(f)
    }()
}

// Panic/Recover — only for truly unexpected states
func safeDivide(a, b int) (result int, err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("panic recovered: %v", r)
        }
    }()
    return a / b, nil  // panics if b==0; recover catches it
}

// HTTP middleware recover pattern
func recoveryMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if r := recover(); r != nil {
                log.Printf("panic: %v
%s", r, debug.Stack())
                http.Error(w, "Internal Server Error", 500)
            }
        }()
        next.ServeHTTP(w, r)
    })
}
```

1.11 Reflection and Generics (Go 1.18+)

```
// Generics — type parameters with constraints
func Map[T, U any](slice []T, f func(T) U) []U {
    result := make([]U, len(slice))
    for i, v := range slice { result[i] = f(v) }
    return result
}

// Constraint with comparable
func Contains[T comparable](slice []T, item T) bool {
    for _, v := range slice {
        if v == item { return true }
    }
    return false
}

// Type constraints
type Number interface {
    ~int | ~int32 | ~int64 | ~float32 | ~float64
}
func Sum[T Number](nums []T) T {
    var total T
    for _, n := range nums { total += n }
    return total
}

// Generic result type
type Result[T any] struct {
    Value T
    Err   error
}
func (r Result[T]) Unwrap() (T, error) { return r.Value, r.Err }

// Reflection — use sparingly (slow, no compile-time safety)
func printStruct(v any) {
    t := reflect.TypeOf(v)
    val := reflect.ValueOf(v)
    for i := 0; i < t.NumField(); i++ {
        field := t.Field(i)
        value := val.Field(i)
        fmt.Printf("%s: %v
", field.Name, value.Interface())
    }
}
```

1.12 Go Profiling: pprof, Trace, Goroutine Leaks

```
// Add pprof endpoint to HTTP server
import _ "net/http/pprof"  // side-effect import registers handlers

go func() {
    log.Println(http.ListenAndServe(":6060", nil))  // separate port
}()

// Capture profiles
go tool pprof http://localhost:6060/debug/pprof/heap       # heap
go tool pprof http://localhost:6060/debug/pprof/goroutine  # goroutines
go tool pprof -seconds 30 http://localhost:6060/debug/pprof/profile  # CPU

# In pprof interactive mode
(pprof) top 10         # top functions by allocation
```

```
(pprof) list funcName  # source-level annotation
(pprof) web            # opens flame graph in browser
(pprof) svg > out.svg  # save flame graph

# Trace tool
curl http://localhost:6060/debug/pprof/trace?seconds=5 > trace.out
go tool trace trace.out   # opens browser visualization

// Benchmark profiling
func BenchmarkHeavy(b *testing.B) {
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        heavyFunction()
    }
}
go test -bench=BenchmarkHeavy -benchmem -cpuprofile=cpu.out -memprofile=mem.out
go tool pprof cpu.out
```

1.13 Writing Testable Code: Unit Tests, Table-Driven, Mocks, Benchmarks

```
// Table-driven tests (idiomatic Go)
func TestAdd(t *testing.T) {
    tests := []struct {
        name     string
        a, b     int
        expected int
    }{
        {"positive", 1, 2, 3},
        {"negative", -1, -2, -3},
        {"zero", 0, 0, 0},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            got := Add(tc.a, tc.b)
            if got != tc.expected {
                t.Errorf("Add(%d,%d) = %d; want %d", tc.a, tc.b, got, tc.expected)
            }
        })
    }
}

// Mocking with interfaces (no framework needed for simple cases)
type MockUserRepo struct {
    users map[int64]*User
}
func (m *MockUserRepo) FindByID(ctx context.Context, id int64) (*User, error) {
    u, ok := m.users[id]
    if !ok { return nil, ErrNotFound }
    return u, nil
}

// Using testify/mock for complex cases
type MockDB struct { mock.Mock }
func (m *MockDB) Query(ctx context.Context, sql string, args ...any) (Rows, error) {
    args2 := m.Called(ctx, sql, args)
    return args2.Get(0).(Rows), args2.Error(1)
}

// Benchmarks
func BenchmarkJSON(b *testing.B) {
    data := generateLargeJSON()
    b.ResetTimer()  // exclude setup time
    b.ReportAllocs()
```

```
    for i := 0; i < b.N; i++ {
        var result MyStruct
        json.Unmarshal(data, &result)
    }
}

// Race detector (CRITICAL in CI)
go test -race ./...
```

1.14 Production Errors — Full Troubleshooting

ERROR SECTION

ERROR: `fatal error: concurrent map read and map write`

ROOT CAUSE: Multiple goroutines reading/writing the same map without synchronization. Go's map is not goroutine-safe.

FIX: Wrap with sync.RWMutex or use sync.Map. Run with -race flag to detect: go test -race ./...

```
// Detection
go build -race ./...
go test -race ./...

// Fix with RWMutex
type SafeMap struct {
    mu   sync.RWMutex
    data map[string]int
}
func (m *SafeMap) Get(k string) int {
    m.mu.RLock(); defer m.mu.RUnlock()
    return m.data[k]
}
func (m *SafeMap) Set(k string, v int) {
    m.mu.Lock(); defer m.mu.Unlock()
    m.data[k] = v
}
```

ERROR: `panic: runtime error: invalid memory address or nil pointer dereference`

ROOT CAUSE: Accessing a field or method on a nil pointer. Common in: unchecked error returns, uninitialized structs, type assertion without ok check.

FIX: Always check errors before using return values. Use defensive nil checks. Enable nil pointer analysis with go vet.

```
// BAD: unchecked error
resp, _ := http.Get(url)
body, _ := io.ReadAll(resp.Body)  // panics if resp is nil

// GOOD:
resp, err := http.Get(url)
if err != nil { return nil, fmt.Errorf("http get: %w", err) }
defer resp.Body.Close()
body, err := io.ReadAll(resp.Body)
```

ERROR: `fatal error: all goroutines are asleep - deadlock!`

ROOT CAUSE: All goroutines are blocked waiting on channels or mutexes with no goroutine able to make progress.

FIX: Use select with default or timeout. Ensure every send has a receiver. Use buffered channels where appropriate.

```
// Deadlock example (unbuffered, no receiver)
```

```
ch := make(chan int)
ch <- 42  // blocks forever

// Fix: buffered channel
ch := make(chan int, 1)
ch <- 42  // doesn't block

// OR: goroutine to receive
go func() { <-ch }()
ch <- 42
```

ERROR: `goroutine leak: N goroutines stuck in chan receive`

ROOT CAUSE: Goroutines blocked on channel receive that never gets data — often from worker pools not receiving shutdown signal.

FIX: Always use context for cancellation. Close channels to unblock range loops. Use goleak in tests.

```
// goleak in tests
func TestWorkerPool(t *testing.T) {
    defer goleak.VerifyNone(t)
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()  // this unblocks workers

    pool := NewWorkerPool(ctx, 10)
    pool.Submit(task)
    // cancel() called by defer → workers exit cleanly
}
```

1.15 Interview Questions

CONCEPTUAL QUESTIONS

Q: Explain the G-M-P goroutine scheduling model.

A: G=goroutine (stack+state), M=OS thread, P=logical processor holding a run queue. GOMAXPROCS controls P count. Scheduler is cooperative+preemptive since Go 1.14 (signal-based preemption at 10ms). When M blocks on syscall, P detaches and finds/creates another M, keeping CPUs busy.

Q: What's the difference between buffered and unbuffered channels? When would you choose each?

A: Unbuffered: synchronous rendezvous — sender blocks until receiver ready. Buffered: async up to capacity. Use unbuffered for synchronization/signaling (guaranteed handshake). Use buffered to smooth bursts, decouple producer/consumer speed, or implement semaphores (make(chan struct{}, N)).

Q: How does Go's garbage collector work and how do you reduce GC pressure?

A: Tri-color concurrent mark-and-sweep GC. Runs concurrently with application (short STW pauses only for root marking and termination). Reduce pressure by: pre-allocating slices (make([]T, 0, n)), using sync.Pool for short-lived objects, avoiding unnecessary allocations in hot paths, setting GOMEMLIMIT in containers.

Q: Explain errors.Is vs errors.As — what's the difference?

A: errors.Is checks error identity — traverses the wrapped error chain looking for a specific sentinel value (==). errors.As checks error type — traverses the chain and assigns to a target pointer if it finds a matching type. Use Is for sentinel errors (ErrNotFound), As for structured error types (PostgresError) to extract fields.

Q: When should you use sync.Mutex vs channels for synchronization?

A: Mutex: protecting shared state (cache, counter, map). Channels: communicating values, signaling events, pipeline patterns. Rob Pike's rule: 'Share memory by communicating.' If data flows between goroutines — channel. If multiple goroutines guard shared state — mutex.

SCENARIO-BASED QUESTIONS

Q: Your service memory keeps growing until OOMKilled. How do you debug and fix it?

A: 1) Add pprof endpoint, capture heap profile: go tool pprof /debug/pprof/heap. 2) Look at top allocations. 3) Common causes: goroutine leak (each has stack), slice references holding large backing arrays, unbounded caches, HTTP response bodies not drained/closed. 4) Fix with sync.Pool, explicit GC hints, bounded caches with eviction, defer resp.Body.Close().

Q: You have a data race reported by -race. Walk me through finding and fixing it.

A: 1) The race detector gives exact file:line for conflicting accesses. 2) Identify the shared variable. 3) If simple counter → use atomic.AddInt64. 4) If struct with multiple fields → sync.Mutex (Lock/Unlock with defer). 5) If high-read/low-write → sync.RWMutex. 6) Confirm fix: re-run go test -race ./...

Q: How would you implement a worker pool with graceful shutdown?

A: Use a buffered jobs channel, N worker goroutines select on {jobs channel, ctx.Done()}. Main goroutine calls cancel() to signal shutdown, then close(jobs) for range-based drain, then wg.Wait(). This ensures in-flight jobs complete before exit.

2. REST API DESIGN

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

REST (Representational State Transfer) is an architectural style for HTTP APIs using resources and standard verbs. It solves the problem of ad-hoc API design. Use it for public-facing APIs, browser-accessible services, and simple CRUD where HTTP semantics suffice. Use gRPC for internal service-to-service when performance matters.

2.1 REST Principles & Resource Naming

- Stateless: every request must contain all info needed; no server-side session
- Resource-based URLs: /users/123/orders, not /getUserOrders?userId=123
- HTTP verbs: GET (read), POST (create), PUT (full replace), PATCH (partial update), DELETE
- Uniform interface, layered system, cacheable responses

```
// Resource naming conventions
GET    /v1/users              // list users
POST   /v1/users              // create user
GET    /v1/users/{id}         // get user
PUT    /v1/users/{id}         // replace user
PATCH  /v1/users/{id}         // update user fields
DELETE /v1/users/{id}         // delete user
GET    /v1/users/{id}/orders  // nested resource

// Pagination
GET /v1/users?page=2&limit=25&sort=created_at:desc

// Filtering
GET /v1/orders?status=pending&from=2024-01-01&to=2024-12-31

// Response structure (consistent across endpoints)
{
  "data": [...],
  "meta": { "page": 2, "limit": 25, "total": 150 },
  "links": { "next": "/v1/users?page=3", "prev": "/v1/users?page=1" }
}
```

2.2 Versioning, Authentication (JWT + OAuth2 in Go)

```
// URL versioning (most common, explicit)
/v1/users, /v2/users

// Header versioning
Accept: application/vnd.myapi.v2+json

// JWT middleware in Go
func JWTMiddleware(secret string) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            authHeader := r.Header.Get("Authorization")
            if !strings.HasPrefix(authHeader, "Bearer ") {
                http.Error(w, "missing bearer token", http.StatusUnauthorized)
                return
            }
            tokenStr := strings.TrimPrefix(authHeader, "Bearer ")
            token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (any, error) {
                if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
                    return nil, fmt.Errorf("unexpected signing method")
```

```
                }
                return []byte(secret), nil
            })
            if err != nil || !token.Valid {
                http.Error(w, "invalid token", http.StatusUnauthorized)
                return
            }
            claims := token.Claims.(jwt.MapClaims)
            ctx := context.WithValue(r.Context(), "userID", claims["sub"])
            next.ServeHTTP(w, r.WithContext(ctx))
        })
    }
}

// Rate limiting with golang.org/x/time/rate
var limiter = rate.NewLimiter(rate.Every(time.Second), 100) // 100 req/s

func rateLimitMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        if !limiter.Allow() {
            http.Error(w, "rate limit exceeded", http.StatusTooManyRequests)
            return
        }
        next.ServeHTTP(w, r)
    })
}
```

2.3 Middleware Chain, Request Validation, Error Standards

```
// Chi middleware chain
r := chi.NewRouter()
r.Use(middleware.RequestID)
r.Use(middleware.Logger)
r.Use(middleware.Recoverer)
r.Use(middleware.Timeout(60 * time.Second))
r.Use(JWTMiddleware(secret))
r.Use(rateLimitMiddleware)

r.Get("/v1/users/{id}", getUserHandler)

// Request validation
type CreateUserRequest struct {
    Email    string `json:"email" validate:"required,email"`
    Name     string `json:"name" validate:"required,min=2,max=100"`
    Age      int    `json:"age" validate:"required,gte=18,lte=120"`
}

var validate = validator.New()

func createUserHandler(w http.ResponseWriter, r *http.Request) {
    var req CreateUserRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        writeError(w, http.StatusBadRequest, "invalid JSON", err.Error())
        return
    }
    if err := validate.Struct(req); err != nil {
        writeError(w, http.StatusBadRequest, "validation failed", err.Error())
        return
    }
    // process...
}

// Standardized error response
```

```go
type ErrorResponse struct {
    Error   string `json:"error"`
    Message string `json:"message"`
    Code    int    `json:"code"`
}

func writeError(w http.ResponseWriter, status int, error, message string) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(ErrorResponse{
        Error: error, Message: message, Code: status,
    })
}
```

2.4 Production Errors — REST API

ERROR SECTION

ERROR: `HTTP 401 vs 403 confusion — frontend shows 'Unauthorized' when user IS authenticated`

ROOT CAUSE: 401 Unauthorized = not authenticated (no/invalid token). 403 Forbidden = authenticated but not authorized (lacks permission). Mixing them confuses clients about whether to retry with credentials.

FIX: Return 401 when token is missing/invalid/expired. Return 403 when token is valid but user lacks permission to access the resource.

ERROR: `Access-Control-Allow-Origin header missing — CORS error in browser`

ROOT CAUSE: Browser blocks cross-origin requests without CORS headers. Server doesn't set Access-Control-Allow-Origin on response.

FIX: Add CORS middleware. Handle OPTIONS preflight requests. Be explicit with allowed origins in production (don't use * with credentials).

```go
// CORS middleware with Chi
import "github.com/go-chi/cors"
r.Use(cors.Handler(cors.Options{
    AllowedOrigins:   []string{"https://myapp.com"},
    AllowedMethods:   []string{"GET", "POST", "PUT", "DELETE", "OPTIONS"},
    AllowedHeaders:   []string{"Accept", "Authorization", "Content-Type"},
    AllowCredentials: true,
    MaxAge:           300,
}))
```

ERROR: `http: request body too large — 413 Payload Too Large`

ROOT CAUSE: Go's net/http defaults to no body size limit. Large file uploads or malformed clients can exhaust memory.

FIX: Set http.MaxBytesReader on every handler or globally in the server.

```go
// Per-handler limit
func uploadHandler(w http.ResponseWriter, r *http.Request) {
    r.Body = http.MaxBytesReader(w, r.Body, 10<<20) // 10MB
    if err := r.ParseMultipartForm(10 << 20); err != nil {
        http.Error(w, "file too large", http.StatusRequestEntityTooLarge)
        return
    }
}

// Server-level timeout
server := &http.Server{
    ReadTimeout:  15 * time.Second,
    WriteTimeout: 15 * time.Second,
    IdleTimeout:  60 * time.Second,
}
```

2.5 Interview Questions — REST API

Q: What is idempotency and which HTTP methods are idempotent?

A: An operation is idempotent if calling it multiple times produces the same result. GET, PUT, DELETE, HEAD are idempotent. POST is not (creates new resource each call). PATCH may or may not be. Important for retries: safe to retry idempotent operations on network failure.

Q: How do you design pagination for 100M record tables?

A: Offset pagination (LIMIT/OFFSET) is O(n) for large offsets — scans all preceding rows. Use cursor-based pagination: encode last-seen ID or timestamp as cursor, query WHERE id > cursor LIMIT n. Faster, consistent under insertions. Return next_cursor in response for client.

Q: How would you rate limit per-user vs global?

A: Use sliding window or token bucket per-user key (IP or user ID) in Redis. Global limiter with golang.org/x/time/rate. Tiered: free users 100 req/min, premium 1000 req/min. Return Retry-After header with 429 response.

3. gRPC

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

gRPC is a high-performance RPC framework by Google using Protocol Buffers (binary serialization) over HTTP/2. It solves REST's loose contract problem and JSON overhead. Use it for internal service-to-service communication where you need strong contracts, streaming, and ~7x lower payload than JSON.

3.1 Protocol Buffers & Service Definition

```
// user.proto
syntax = "proto3";
package user.v1;
option go_package = "github.com/myco/myservice/gen/user/v1;userv1";

import "google/protobuf/timestamp.proto";

message User {
    int64  id         = 1;
    string email      = 2;
    string name       = 3;
    google.protobuf.Timestamp created_at = 4;
}

message GetUserRequest  { int64 id    = 1; }
message GetUserResponse { User  user  = 1; }

message ListUsersRequest  { int32 page = 1; int32 limit = 2; }
message ListUsersResponse { repeated User users = 1; int32 total = 2; }

// Streaming: server pushes real-time events
message SubscribeRequest  { int64 user_id = 1; }
message UserEvent         { string type = 1; User user = 2; }

service UserService {
    rpc GetUser(GetUserRequest) returns (GetUserResponse);
    rpc ListUsers(ListUsersRequest) returns (ListUsersResponse);
    // Server streaming
    rpc SubscribeEvents(SubscribeRequest) returns (stream UserEvent);
    // Client streaming
    rpc BatchCreateUsers(stream User) returns (ListUsersResponse);
    // Bidirectional streaming
    rpc Chat(stream ChatMessage) returns (stream ChatMessage);
}

// Code generation
protoc --go_out=./gen --go_opt=paths=source_relative \
       --go-grpc_out=./gen --go-grpc_opt=paths=source_relative \
       proto/user/v1/user.proto
```

3.2 gRPC Server Implementation in Go

```
// Implementing the server
type userServer struct {
    userv1.UnimplementedUserServiceServer  // forward compatibility
    repo UserRepository
}
```

```go
func (s *userServer) GetUser(ctx context.Context, req *userv1.GetUserRequest)
(*userv1.GetUserResponse, error) {
    if req.Id <= 0 {
        return nil, status.Errorf(codes.InvalidArgument, "invalid user id: %d", req.Id)
    }
    user, err := s.repo.FindByID(ctx, req.Id)
    if errors.Is(err, ErrNotFound) {
        return nil, status.Errorf(codes.NotFound, "user %d not found", req.Id)
    }
    if err != nil {
        return nil, status.Errorf(codes.Internal, "get user: %v", err)
    }
    return &userv1.GetUserResponse{User: toProto(user)}, nil
}

// Server streaming
func (s *userServer) SubscribeEvents(req *userv1.SubscribeRequest, stream
userv1.UserService_SubscribeEventsServer) error {
    events := s.eventBus.Subscribe(req.UserId)
    defer s.eventBus.Unsubscribe(req.UserId, events)

    for {
        select {
        case event := <-events:
            if err := stream.Send(toEventProto(event)); err != nil {
                return err  // client disconnected
            }
        case <-stream.Context().Done():
            return nil  // client cancelled
        }
    }
}

// Starting the server
func main() {
    lis, err := net.Listen("tcp", ":9090")
    if err != nil { log.Fatal(err) }

    s := grpc.NewServer(
        grpc.UnaryInterceptor(grpc_middleware.ChainUnaryServer(
            grpc_auth.UnaryServerInterceptor(authFunc),
            grpc_zap.UnaryServerInterceptor(logger),
            grpc_recovery.UnaryServerInterceptor(),
        )),
    )
    userv1.RegisterUserServiceServer(s, &userServer{repo: repo})
    reflection.Register(s)  // enables grpcurl and grpc-gateway

    log.Fatal(s.Serve(lis))
}
```

3.3 gRPC Interceptors, Metadata, Deadlines

```go
// Interceptor (middleware) — logging + auth
func loggingInterceptor(ctx context.Context, req any, info *grpc.UnaryServerInfo, handler
grpc.UnaryHandler) (any, error) {
    start := time.Now()
    resp, err := handler(ctx, req)
    log.Printf("method=%s duration=%s err=%v", info.FullMethod, time.Since(start), err)
    return resp, err
}

// Metadata (headers in gRPC)
```

```go
// Client side
ctx := metadata.AppendToOutgoingContext(ctx,
    "authorization", "Bearer "+token,
    "x-request-id", requestID,
)

// Server side
md, ok := metadata.FromIncomingContext(ctx)
if ok {
    vals := md.Get("x-request-id")
    if len(vals) > 0 { requestID = vals[0] }
}

// Deadlines propagation
ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
defer cancel()

client := userv1.NewUserServiceClient(conn)
resp, err := client.GetUser(ctx, &userv1.GetUserRequest{Id: 123})
if status.Code(err) == codes.DeadlineExceeded {
    log.Println("request timed out")
}
```

3.4 Production Errors — gRPC

ERROR SECTION

ERROR: `rpc error: code = Unavailable desc = connection refused`

ROOT CAUSE: Service not running, wrong address/port, or network policy blocking connection.

FIX: Verify service is up (kubectl get pods). Check address config. Add retry with backoff: grpc.WithDefaultServiceConfig(`{"methodConfig": [{"retryPolicy": {...}}]}`).

ERROR: `rpc error: code = DeadlineExceeded desc = context deadline exceeded`

ROOT CAUSE: Client-set deadline elapsed before server responded. Can be slow DB query, network latency, or chain of calls not propagating remaining deadline.

FIX: Always propagate ctx through call chain. Set per-method timeouts appropriate to SLA. Add tracing to find which downstream is slow.

ERROR: `rpc error: code = ResourceExhausted desc = grpc: received message larger than max`

ROOT CAUSE: Message exceeds 4MB default limit. Common with large batch responses.

FIX: Set MaxRecvMsgSize on server, MaxSendMsgSize on client. Or redesign to stream large payloads instead of single message.

```go
grpc.NewServer(
    grpc.MaxRecvMsgSize(16*1024*1024), // 16MB
    grpc.MaxSendMsgSize(16*1024*1024),
)

conn, err := grpc.Dial(addr,
    grpc.WithDefaultCallOptions(
        grpc.MaxCallRecvMsgSize(16*1024*1024),
    ),
)
```

ERROR: `transport: authentication handshake failed: tls: certificate signed by unknown authority`

ROOT CAUSE: Client has TLS enabled but server cert not trusted, or vice versa.

FIX: Load proper CA cert on client side. Or use insecure.NewCredentials() in dev (never prod).

3.5 Interview Questions — gRPC

Q: When would you choose gRPC over REST?

A: gRPC: internal microservice-to-microservice (binary efficient, strong contract via .proto, bi-directional streaming, generated client/server code, deadline propagation). REST: public API (human-readable, browser-native, wide tooling), simple CRUD, when HTTP semantics are sufficient.

Q: Explain the 4 gRPC communication patterns.

A: 1) Unary: single request/response (like REST). 2) Server Streaming: client sends one request, server sends stream (real-time feed, logs). 3) Client Streaming: client sends stream, server replies once (batch upload, sensor data). 4) Bidirectional Streaming: both stream simultaneously (chat, real-time collaborative editing).

Q: How do gRPC error codes map to HTTP status codes?

A: OK=200, InvalidArgument=400, Unauthenticated=401, PermissionDenied=403, NotFound=404, AlreadyExists=409, ResourceExhausted=429, Internal=500, Unavailable=503, DeadlineExceeded=504. Always use semantic gRPC codes, not wrapping arbitrary messages in Internal.

4. SYSTEM DESIGN & DISTRIBUTED SYSTEMS

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

Distributed systems design addresses building reliable, scalable software across multiple machines. It involves fundamental tradeoffs (consistency vs availability), patterns for failure handling (circuit breakers, sagas), and architectural choices (event sourcing, CQRS).

4.1 CAP Theorem

In a network partition, you must choose between Consistency and Availability. You cannot have all three simultaneously.

- CP systems (Consistency + Partition Tolerance): ZooKeeper, etcd, HBase — choose correctness over availability. Used for: leader election, distributed locks, configuration.
- AP systems (Availability + Partition Tolerance): Cassandra, CouchDB, DynamoDB — always respond, possibly with stale data. Used for: shopping carts, social feeds, DNS.
- CA systems: Traditional RDBMS in single node — partition tolerance sacrificed, so not suitable for distributed deployment.

> **Real Example:** Cassandra (AP): during partition, all nodes respond with potentially stale data. Tunable consistency: quorum reads give stronger consistency at cost of availability.

4.2 Load Balancing: L4 vs L7, Algorithms

```
// L4 (Transport Layer): routes by IP/TCP — fast, no HTTP inspection
// L7 (Application Layer): routes by HTTP headers, URL, cookies — intelligent routing

// Algorithms:
// Round Robin: requests distributed evenly (ignores server load)
// Least Connections: next request to server with fewest active connections
// Consistent Hashing: same key always hits same server (good for caching)
//   ring = hash(serverID) -> position; request -> nearest server on ring

// Consistent hashing in Go
type Ring struct {
    nodes    map[uint32]string
    sorted   []uint32
}

func (r *Ring) Add(node string) {
    for i := 0; i < 100; i++ {  // 100 virtual nodes per server
        hash := crc32.ChecksumIEEE([]byte(fmt.Sprintf("%s-%d", node, i)))
        r.nodes[hash] = node
        r.sorted = append(r.sorted, hash)
    }
    sort.Slice(r.sorted, func(i, j int) bool { return r.sorted[i] < r.sorted[j] })
}

func (r *Ring) Get(key string) string {
    hash := crc32.ChecksumIEEE([]byte(key))
    idx := sort.Search(len(r.sorted), func(i int) bool { return r.sorted[i] >= hash })
    if idx == len(r.sorted) { idx = 0 }  // wrap around ring
    return r.nodes[r.sorted[idx]]
}
```

4.3 Circuit Breaker Pattern

```go
// Circuit breaker states: Closed (normal), Open (failing), Half-Open (testing)
import "github.com/sony/gobreaker"

var cb = gobreaker.NewCircuitBreaker(gobreaker.Settings{
    Name:        "downstream-service",
    MaxRequests: 3,     // requests allowed in half-open state
    Interval:    10 * time.Second,
    Timeout:     30 * time.Second,  // time to wait before half-open
    ReadyToTrip: func(counts gobreaker.Counts) bool {
        // Open circuit if >50% failure rate with at least 5 requests
        return counts.ConsecutiveFailures > 5 ||
            float64(counts.TotalFailures)/float64(counts.Requests) > 0.5
    },
})

func callDownstream(ctx context.Context, req Request) (Response, error) {
    result, err := cb.Execute(func() (any, error) {
        return client.Call(ctx, req)
    })
    if err == gobreaker.ErrOpenState {
        return Response{}, fmt.Errorf("circuit open, downstream unavailable")
    }
    return result.(Response), err
}
```

4.4 Rate Limiting Algorithms

```go
// Token Bucket: bucket holds N tokens, refills at rate R
// Allows bursting (use all tokens instantly)
// Go: golang.org/x/time/rate uses token bucket
limiter := rate.NewLimiter(rate.Every(time.Second/100), 10) // 100/s, burst 10
if !limiter.Allow() { return http.StatusTooManyRequests }

// Leaky Bucket: requests queue up and drain at fixed rate (no bursting)
// Useful for smooth traffic shaping to downstream

// Sliding Window: count requests in last N seconds using Redis sorted set
func isRateLimited(ctx context.Context, userID string, limit int) bool {
    now := time.Now().UnixMilli()
    window := int64(60 * 1000) // 60 second window
    key := "rate:" + userID

    pipe := rdb.Pipeline()
    pipe.ZRemRangeByScore(ctx, key, "0", strconv.FormatInt(now-window, 10))
    pipe.ZCard(ctx, key)
    pipe.ZAdd(ctx, key, &redis.Z{Score: float64(now), Member: now})
    pipe.Expire(ctx, key, 70*time.Second)
    results, _ := pipe.Exec(ctx)

    count := results[1].(*redis.IntCmd).Val()
    return count >= int64(limit)
}
```

4.5 Saga Pattern for Distributed Transactions

Distributed transactions can't use ACID across services. Saga splits into local transactions with compensating actions on failure.

```go
// Choreography Saga — services react to events
// Order created → Payment attempted → Inventory reserved → Ship
```

```go
// On payment failure → OrderCancelled event → Inventory released

// Orchestration Saga — central coordinator
type OrderSaga struct {
    orderSvc     OrderService
    paymentSvc   PaymentService
    inventorySvc InventoryService
}

func (s *OrderSaga) Execute(ctx context.Context, order Order) error {
    // Step 1: Reserve inventory
    if err := s.inventorySvc.Reserve(ctx, order.Items); err != nil {
        return fmt.Errorf("reserve inventory: %w", err)
    }

    // Step 2: Charge payment (compensate inventory on failure)
    if err := s.paymentSvc.Charge(ctx, order.PaymentInfo); err != nil {
        s.inventorySvc.Release(ctx, order.Items)  // compensating transaction
        return fmt.Errorf("charge payment: %w", err)
    }

    // Step 3: Confirm order
    if err := s.orderSvc.Confirm(ctx, order.ID); err != nil {
        s.paymentSvc.Refund(ctx, order.PaymentInfo)   // compensate
        s.inventorySvc.Release(ctx, order.Items)       // compensate
        return fmt.Errorf("confirm order: %w", err)
    }
    return nil
}
```

4.6 IoT Data Ingestion Pipeline Design

This is highly relevant to the job description. Design for 100k+ sensors sending data every second.

- Ingestion Layer: MQTT broker or Kafka — sensors push to partitioned topics by device_id
- Stream Processing: Go consumers per partition, batching writes to DB (1000 events/batch, 500ms flush interval)
- Storage: TimescaleDB (PostgreSQL time-series extension) or InfluxDB for raw metrics; PostgreSQL for device registry
- Fan-out: consumer groups for analytics, alerting, archival

```go
// High-throughput IoT consumer pattern
type IoTConsumer struct {
    consumer  kafka.Reader
    db        *pgxpool.Pool
    batchSize int
    flushInterval time.Duration
}

func (c *IoTConsumer) Run(ctx context.Context) error {
    batch := make([]SensorReading, 0, c.batchSize)
    ticker := time.NewTicker(c.flushInterval)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return c.flush(ctx, batch)
        case <-ticker.C:
            if len(batch) > 0 {
                if err := c.flush(ctx, batch); err != nil { return err }
                batch = batch[:0]
```

```go
            }
        default:
            msg, err := c.consumer.ReadMessage(ctx)
            if err != nil { return err }

            var reading SensorReading
            if err := json.Unmarshal(msg.Value, &reading); err != nil { continue }
            batch = append(batch, reading)

            if len(batch) >= c.batchSize {
                if err := c.flush(ctx, batch); err != nil { return err }
                batch = batch[:0]
            }
        }
    }
}

func (c *IoTConsumer) flush(ctx context.Context, batch []SensorReading) error {
    // COPY FROM for bulk insert (10x faster than INSERT)
    _, err := c.db.CopyFrom(ctx,
        pgx.Identifier{"sensor_readings"},
        []string{"device_id", "metric", "value", "ts"},
        pgx.CopyFromSlice(len(batch), func(i int) ([]any, error) {
            r := batch[i]
            return []any{r.DeviceID, r.Metric, r.Value, r.Timestamp}, nil
        }),
    )
    return err
}
```

4.7 Interview Questions — System Design

Q: Explain split-brain and how to prevent it.

A: Split-brain: network partition creates two independent clusters that both believe they're the primary (e.g., two DB primaries accepting writes). Prevention: quorum/majority consensus (Raft, Paxos) — a node only acts as primary with votes from >N/2 nodes. etcd and PostgreSQL synchronous replication use this. Also: STONITH (Shoot The Other Node In The Head) — fence the minority partition.

Q: Design a real-time leaderboard for 10M users.

A: Redis Sorted Set: ZADD leaderboard <score> <userID>, ZREVRANK for rank, ZREVRANGE for top-N. O(log N) for all operations. For user's rank: ZREVRANK + 1. Periodically snapshot to PostgreSQL for durability. Cache top-100 in-memory with 1s TTL. Handle hot keys by sharding by game region.

Q: What is the thundering herd problem and how do you solve it?

*A: When a cache expires, all concurrent requests hit the DB simultaneously (cache stampede). Solutions: 1) Cache lock: first request acquires Redis lock, computes value, others wait. 2) Probabilistic early expiration (refresh before TTL ends). 3) Stale-while-revalidate: serve stale while refreshing in background. 4) Jitter on TTL: TTL = base + random(0, base*0.1).*

5. POSTGRESQL

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

PostgreSQL is a production-grade open-source RDBMS with full ACID compliance, advanced indexing, JSONB, and rich extension ecosystem. Use it as your primary store for relational data, audit logs, financial transactions, and when you need strong consistency.

5.1 ACID Properties with Go Transaction Examples

```go
// Atomicity: all or nothing
// Consistency: data always valid
// Isolation: concurrent transactions appear sequential
// Durability: committed data survives crashes

// Go transaction with pgx
func (r *repo) TransferFunds(ctx context.Context, fromID, toID int64, amount
decimal.Decimal) error {
    tx, err := r.pool.Begin(ctx)
    if err != nil { return fmt.Errorf("begin tx: %w", err) }
    defer tx.Rollback(ctx)  // no-op if committed

    // Lock rows in consistent order to prevent deadlock
    if fromID > toID { fromID, toID = toID, fromID }

    var fromBalance decimal.Decimal
    err = tx.QueryRow(ctx,
        "SELECT balance FROM accounts WHERE id = $1 FOR UPDATE",
        fromID).Scan(&fromBalance)
    if err != nil { return fmt.Errorf("get from balance: %w", err) }

    if fromBalance.LessThan(amount) {
        return ErrInsufficientFunds
    }

    _, err = tx.Exec(ctx,
        "UPDATE accounts SET balance = balance - $1 WHERE id = $2",
        amount, fromID)
    if err != nil { return fmt.Errorf("debit: %w", err) }

    _, err = tx.Exec(ctx,
        "UPDATE accounts SET balance = balance + $1 WHERE id = $2",
        amount, toID)
    if err != nil { return fmt.Errorf("credit: %w", err) }

    if err := tx.Commit(ctx); err != nil {
        return fmt.Errorf("commit: %w", err)
    }
    return nil
}
```

5.2 Index Types — When to Use Each

- B-tree (default): equality and range queries, ORDER BY, most common case
- GIN (Generalized Inverted Index): JSONB containment (@>), array operators, full-text search (tsvector)
- GiST (Generalized Search Tree): geometric data, PostGIS spatial queries, range types

- BRIN (Block Range INdex): very large tables with natural sort correlation (timestamps, sequential IDs) — tiny index size

```
-- B-tree (most common)
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_orders_user_created ON orders(user_id, created_at DESC);

-- Partial index (only index relevant rows)
CREATE INDEX idx_orders_pending ON orders(created_at) WHERE status = 'pending';

-- GIN for JSONB
CREATE INDEX idx_devices_metadata ON devices USING GIN(metadata);
-- Query: SELECT * FROM devices WHERE metadata @> '{"type": "sensor"}';

-- GIN for full-text search
CREATE INDEX idx_products_fts ON products USING GIN(to_tsvector('english', name || ' ' ||
description));
-- Query: SELECT * FROM products WHERE to_tsvector('english', name) @@ to_tsquery('golang
& backend');

-- BRIN for time-series (sensor_readings has billions of rows)
CREATE INDEX idx_readings_ts ON sensor_readings USING BRIN(recorded_at);
```

5.3 EXPLAIN ANALYZE — Reading Query Plans

```
-- Always use EXPLAIN (ANALYZE, BUFFERS) for real execution data
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT u.name, COUNT(o.id) as order_count
FROM users u
JOIN orders o ON o.user_id = u.id
WHERE o.created_at > NOW() - INTERVAL '30 days'
GROUP BY u.id, u.name
ORDER BY order_count DESC
LIMIT 10;

-- Reading the plan:
-- Seq Scan: reading entire table — usually bad on large tables, add index
-- Index Scan: using index — good
-- Index Only Scan: covers all needed columns — best
-- Nested Loop: good for small inner sets
-- Hash Join: good for larger sets
-- Merge Join: both sides sorted
-- Rows=X (actual rows=Y): large discrepancy = stale statistics → ANALYZE table
-- Buffers: hit=X read=Y — high read means disk IO, check shared_buffers

-- Key metrics to look for:
-- actual time: wall clock time for that node
-- loops: how many times that node executed
-- Planning Time vs Execution Time
```

5.4 N+1 Problem & Connection Pooling

```
// N+1 problem: 1 query for list + N queries for each item's related data
// BAD: N+1
users, _ := db.Query("SELECT id, name FROM users LIMIT 100")  // 1 query
for _, u := range users {
    orders, _ := db.Query("SELECT * FROM orders WHERE user_id = $1", u.ID) // N queries!
}
```

```
// GOOD: JOIN or IN clause
rows, _ := db.Query(`
    SELECT u.id, u.name, o.id as order_id, o.total
    FROM users u
    LEFT JOIN orders o ON o.user_id = u.id
    WHERE u.id = ANY($1)
`, userIDs)

// pgxpool configuration
pool, err := pgxpool.New(ctx, connString)
config, _ := pgxpool.ParseConfig(connString)
config.MaxConns = 25                        // per instance; total = instances * MaxConns
config.MinConns = 5                         // keep warm connections
config.MaxConnLifetime = 1 * time.Hour
config.MaxConnIdleTime = 30 * time.Minute
config.HealthCheckPeriod = 1 * time.Minute
pool, err = pgxpool.NewWithConfig(ctx, config)
```

5.5 Production Errors — PostgreSQL

ERROR SECTION

ERROR: `ERROR: deadlock detected — DETAIL: Process 1234 waits for ShareLock on transaction 5678`

ROOT CAUSE: Two transactions lock rows in opposite order: T1 locks A then B, T2 locks B then A → circular wait.

FIX: Always acquire locks in consistent order. Use NOWAIT or SET lock_timeout. Retry on deadlock (idempotent operations).

```
// Retry on deadlock in Go
const maxRetries = 3
for i := 0; i < maxRetries; i++ {
    err = transferFunds(ctx, tx)
    if err == nil { break }
    var pgErr *pgconn.PgError
    if errors.As(err, &pgErr) && pgErr.Code == "40P01" { // deadlock_detected
        time.Sleep(time.Duration(i+1) * 50 * time.Millisecond)
        continue
    }
    break
}
```

ERROR: `FATAL: remaining connection slots are reserved for non-replication superuser connections`

ROOT CAUSE: max_connections (default 100) reached. Too many app instances or connection pool misconfigured.

FIX: Use PgBouncer as connection pooler (transaction mode). Reduce MaxConns per app instance. max_connections should be ~4x CPU cores.

ERROR: `ERROR: could not serialize access due to concurrent update`

ROOT CAUSE: Using SERIALIZABLE isolation level — a transaction was aborted to prevent anomaly. Application must retry.

FIX: Implement retry logic for serialization failures (code 40001). Or use explicit locking (SELECT FOR UPDATE) with REPEATABLE READ.

ERROR: `Slow query: Seq Scan on 100M row table taking 30s`

ROOT CAUSE: Missing index, stale statistics, or query that can't use existing index (e.g., function on indexed column).

FIX: Run EXPLAIN ANALYZE. Add appropriate index. Run ANALYZE table to update statistics. Avoid functions on indexed columns in WHERE: use created_at > X not date(created_at) > X.

5.6 Interview Questions — PostgreSQL

Q: Explain the difference between B-tree and GIN indexes.

A: B-tree: balanced tree for ordered comparisons (=, <, >, BETWEEN, LIKE prefix). Single value per row. GIN: inverted index — maps element values to row locations. Used when a row contains multiple searchable items (JSONB keys, array elements, tsvector lexemes). GIN is larger and slower to update but essential for containment queries.

Q: How do you do zero-downtime migrations in production?

A: Principle: never lock the table. Pattern: 1) Add nullable column (no lock). 2) Backfill in batches (UPDATE WHERE id BETWEEN X AND Y). 3) Add NOT NULL constraint with a DEFAULT (lock-free in PG 11+). 4) Create indexes CONCURRENTLY (no lock). 5) Drop old column. For rename: add new column, dual-write, backfill, switch reads, drop old.

Q: What is MVCC and how does it affect vacuuming?

A: MVCC (Multi-Version Concurrency Control): readers never block writers. Old row versions are kept until no transaction needs them. VACUUM reclaims dead tuples. Bloat occurs if autovacuum can't keep up (long transactions hold snapshots). Monitor pg_stat_user_tables.n_dead_tup and track table bloat. Tune autovacuum_vacuum_scale_factor for high-churn tables.

6. MICROSERVICES ARCHITECTURE

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

Microservices decompose a monolith into small, independently deployable services each owning their domain. Solves: monolith deployment coupling, scaling bottlenecks, team autonomy. The cost: distributed systems complexity, network failures, eventual consistency.

6.1 Monolith vs Microservices — Honest Tradeoffs

- Monolith advantages: simpler debugging (single process, stack trace), lower latency (in-process calls), easier transactions, less infrastructure
- Monolith disadvantages: deployment coupling (one bug = full redeploy), scaling is all-or-nothing, technology lock-in
- Microservices advantages: independent deployment, team autonomy, polyglot, targeted scaling
- Microservices disadvantages: distributed system complexity, network failures, distributed transactions, need for observability, higher ops burden

> **Senior Engineer Perspective:** Start with a monolith. Extract services when you have clear seams, team scaling pressure, or specific scaling requirements. The Strangler Fig pattern: incrementally extract from monolith, routing traffic to new service.

6.2 Distributed Tracing with OpenTelemetry in Go

```go
// Setup OpenTelemetry with Jaeger exporter
func initTracer() (*sdktrace.TracerProvider, error) {
    exp, err := jaeger.New(jaeger.WithCollectorEndpoint(
        jaeger.WithEndpoint("http://jaeger:14268/api/traces"),
    ))
    if err != nil { return nil, err }

    tp := sdktrace.New(
        sdktrace.WithBatcher(exp),
        sdktrace.WithResource(resource.NewWithAttributes(
            semconv.SchemaURL,
            semconv.ServiceName("order-service"),
            semconv.ServiceVersion("v1.2.3"),
        )),
    )
    otel.SetTracerProvider(tp)
    otel.SetTextMapPropagator(propagation.TraceContext{})  // W3C trace context
    return tp, nil
}

// Instrument your service
var tracer = otel.Tracer("order-service")

func (s *Service) CreateOrder(ctx context.Context, req CreateOrderRequest) error {
    ctx, span := tracer.Start(ctx, "CreateOrder",
        trace.WithAttributes(
            attribute.String("order.user_id", req.UserID),
            attribute.Int("order.items_count", len(req.Items)),
        ),
    )
    defer span.End()

    if err := s.validateOrder(ctx, req); err != nil {
        span.RecordError(err)
        span.SetStatus(codes.Error, err.Error())
        return err
```

```
    }

    // Span propagates through gRPC/HTTP via context
    return s.repo.Save(ctx, req)
}

// HTTP middleware to extract incoming trace context
otelhttp.NewHandler(handler, "server",
    otelhttp.WithTracerProvider(otel.GetTracerProvider()),
)
```

6.3 12-Factor App Principles for Go Microservices

1. Codebase: one repo per service, many deploys
2. Dependencies: go.mod — explicitly declare all dependencies
3. Config: environment variables (never hardcode); use os.Getenv or viper
4. Backing services: DB, Redis, Kafka as attached resources via config URL
5. Build/release/run: separate stages — binary is immutable artifact
6. Processes: stateless — state in Redis/DB, not in-memory
7. Port binding: service exposes via port; container maps it
8. Concurrency: scale via goroutines and replica pods
9. Disposability: fast startup, graceful shutdown (SIGTERM handler)
10. Dev/prod parity: Docker Compose mirrors prod topology
11. Logs: write to stdout; platform aggregates (no log files in container)
12. Admin processes: use Kubernetes Jobs for migrations

```
// Graceful shutdown
func main() {
    server := &http.Server{Addr: ":8080", Handler: router}

    go func() { server.ListenAndServe() }()

    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
    <-quit

    ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()

    if err := server.Shutdown(ctx); err != nil {
        log.Fatal("forced shutdown:", err)
    }
    log.Println("server exited cleanly")
}
```

6.4 Interview Questions — Microservices

Q: How do you handle a cascading failure when service A calls B which calls C?

A: 1) Timeouts: every call has a deadline shorter than caller's deadline. 2) Circuit breaker: open after threshold failures, fail-fast. 3) Bulkhead: separate thread pools/semaphores per downstream. 4) Fallback: serve cached/default response when circuit open. 5) Retry with exponential backoff + jitter only for idempotent operations.

Q: How do you maintain data consistency across microservices without distributed transactions?

A: Saga pattern: choreography (events) or orchestration (coordinator) with compensating transactions. Outbox pattern: write event to DB in same transaction as business data, separate process reads and publishes to Kafka. Eventual consistency: accept that services will be temporarily out of sync and design UX accordingly.

Q: What is the Outbox pattern and why is it important?

A: Problem: after saving to DB, publishing to Kafka can fail — inconsistency. Outbox: write a row to an events table in the same DB transaction as the business data. A separate process (Debezium or polling) reads the outbox and publishes to Kafka with at-least-once delivery. Marks rows as published. Guarantees event is always published if business data is saved.

7. DOCKER

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

Docker packages applications with their dependencies into portable containers. Solves 'works on my machine.' Use Docker for consistent dev/test/prod environments, image-based deployments, and microservice isolation.

7.1 Multi-Stage Build for Go (Optimized)

```
# Production-grade multi-stage Go Dockerfile
# Stage 1: Build
FROM golang:1.22-alpine AS builder

WORKDIR /app

# Layer caching: copy mod files first (changes rarely)
COPY go.mod go.sum ./
RUN go mod download

# Build binary
COPY . .
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build \
    -ldflags="-w -s" \
    -o /app/server ./cmd/server

# Stage 2: Minimal production image
FROM scratch  # or gcr.io/distroless/static-debian12

# Security: non-root user (scratch doesn't have useradd)
# With distroless:
# USER nonroot:nonroot

# Copy CA certificates for HTTPS calls
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/

# Copy binary only
COPY --from=builder /app/server /server

EXPOSE 8080

ENTRYPOINT ["/server"]

# Result: <20MB image vs 800MB+ with full Go base
```

7.2 Docker Compose for Local Microservices

```
# docker-compose.yml
version: '3.9'
services:
  api:
    build: .
    ports: ["8080:8080"]
    environment:
      - DATABASE_URL=postgres://user:pass@postgres:5432/mydb
      - KAFKA_BROKERS=kafka:9092
      - REDIS_URL=redis:6379
    depends_on:
      postgres: { condition: service_healthy }
      kafka:    { condition: service_healthy }
```

```
    healthcheck:
      test: ["CMD", "wget", "-q", "-O-", "http://localhost:8080/health"]
      interval: 10s
      timeout: 5s
      retries: 5

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./migrations:/docker-entrypoint-initdb.d  # auto-runs on init
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d mydb"]
      interval: 5s
      retries: 10

  redis:
    image: redis:7-alpine
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
    depends_on: [zookeeper]

volumes:
  pgdata:
```

7.3 Production Errors — Docker

ERROR SECTION

ERROR: `exec /server: exec format error`

ROOT CAUSE: Binary compiled for wrong architecture. e.g., built on ARM64 Mac (M1/M2) but running in amd64 container.

FIX: Cross-compile explicitly: GOARCH=amd64 GOOS=linux go build. Or use Docker buildx for multi-arch.

```
# Fix: explicit target arch in Dockerfile
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o /app/server .

# Or build with buildx for multi-platform
docker buildx build --platform linux/amd64,linux/arm64 -t myimage:latest --push .
```

ERROR: `OOMKilled — container killed, Exit Code 137`

ROOT CAUSE: Container exceeded memory limit. Kubernetes or Docker killed it.

FIX: Profile memory with pprof. Set appropriate limits. Check for goroutine/memory leaks. Set GOGC and GOMEMLIMIT appropriately.

ERROR: `Error response from daemon: driver failed programming external connectivity: Bind for 0.0.0.0:5432 failed: port is already allocated`

ROOT CAUSE: Port 5432 already used by another container or host process (local PostgreSQL).

FIX: Stop conflicting service: sudo service postgresql stop. Or map to different host port: '5433:5432' in docker-compose.

ERROR: `permission denied: open /data/file.txt`

ROOT CAUSE: Container runs as non-root but volume owned by root. Or host bind mount has wrong permissions.
FIX: Match UID: run container with --user $(id -u):$(id -g). Or chown in Dockerfile before switching user.

7.4 Interview Questions — Docker

Q: Explain Docker layer caching and how to optimize it for Go.

A: Each Dockerfile instruction creates a layer. Layers are cached by content hash. If a layer changes, all subsequent layers invalidate. For Go: copy go.mod/go.sum first, run go mod download (rarely changes), THEN copy source code (changes frequently). This prevents re-downloading modules on every source change — saves 60-90s per build.

Q: What's the difference between ENTRYPOINT and CMD?

A: ENTRYPOINT: defines the executable; can't be overridden easily (--entrypoint flag). CMD: default arguments to ENTRYPOINT, or default command if no ENTRYPOINT. Best practice: ENTRYPOINT ["/app/server"] CMD ["--config=/etc/config.yaml"]. Running docker run myimage --config=other.yaml overrides CMD, not ENTRYPOINT.

Q: How do you handle secrets in Docker/Kubernetes?

A: Never in Dockerfile (history visible) or docker-compose.yml committed to git. Options: 1) Environment variables from .env (not committed). 2) Docker secrets (Swarm). 3) Kubernetes Secrets + mounted as files. 4) External secret manager: HashiCorp Vault, AWS Secrets Manager with sidecar/init container. 5) Sealed Secrets for GitOps.

8. APACHE KAFKA

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

Kafka is a distributed log — an append-only, partitioned, replicated commit log. It solves durable, high-throughput async messaging, event sourcing, and stream processing. Use it for IoT event ingestion, audit logs, event-driven microservices, and data pipelines. Use NATS for lower-latency messaging where persistence is less critical.

8.1 Core Architecture: Brokers, Topics, Partitions, Offsets

- Broker: a Kafka server. A cluster has N brokers (typically 3 for prod)
- Topic: logical category. Data is ordered within a partition, not across
- Partition: unit of parallelism and ordering. More partitions = more consumer parallelism
- Offset: monotonically increasing position of a message in a partition
- Consumer Group: multiple consumers sharing work. Each partition assigned to ONE consumer in the group
- ISR (In-Sync Replicas): replicas fully caught up to leader. acks=all requires all ISR to acknowledge

```go
// Kafka with segmentio/kafka-go (pure Go, no CGo required)
import "github.com/segmentio/kafka-go"

// Producer
func newProducer(brokers []string, topic string) *kafka.Writer {
    return &kafka.Writer{
        Addr:         kafka.TCP(brokers...),
        Topic:        topic,
        Balancer:     &kafka.LeastBytes{},   // or Hash for key-based partitioning
        RequiredAcks: kafka.RequireAll,       // acks=all for durability
        Async:        false,                  // synchronous for exactly-once guarantee
        Compression:  kafka.Snappy,
        BatchSize:    100,
        BatchTimeout: 10 * time.Millisecond,
    }
}

func publishEvent(ctx context.Context, w *kafka.Writer, event Event) error {
    val, _ := json.Marshal(event)
    return w.WriteMessages(ctx, kafka.Message{
        Key:   []byte(event.DeviceID),  // same device → same partition → ordered
        Value: val,
        Headers: []kafka.Header{
            {Key: "event-type", Value: []byte(event.Type)},
            {Key: "schema-version", Value: []byte("1")},
        },
    })
}
```

8.2 Consumer Groups & Exactly-Once Semantics

```go
// Consumer with manual offset commit
func newConsumer(brokers []string, topic, groupID string) *kafka.Reader {
    return kafka.NewReader(kafka.ReaderConfig{
        Brokers:        brokers,
        Topic:          topic,
        GroupID:        groupID,
```

```go
        MinBytes:       10e3,         // 10KB
        MaxBytes:       10e6,         // 10MB
        MaxWait:        1 * time.Second,
        StartOffset:    kafka.FirstOffset,  // auto.offset.reset = earliest
        CommitInterval: 0,            // manual commit
    })
}

func consumeMessages(ctx context.Context, r *kafka.Reader) error {
    for {
        msg, err := r.FetchMessage(ctx)  // fetch without commit
        if err != nil {
            if errors.Is(err, context.Canceled) { return nil }
            return fmt.Errorf("fetch: %w", err)
        }

        if err := processMessage(ctx, msg); err != nil {
            // Don't commit on processing error — message will be redelivered
            log.Printf("process error, will retry: %v", err)
            continue
        }

        // Commit only after successful processing (at-least-once)
        if err := r.CommitMessages(ctx, msg); err != nil {
            return fmt.Errorf("commit: %w", err)
        }
    }
}

// Exactly-once: use Kafka transactions (idempotent producer + transactional consumer)
// OR: use database-level idempotency key
// INSERT INTO processed_events (offset, partition, payload)
//   ON CONFLICT (offset, partition) DO NOTHING
```

8.3 Kafka for IoT — High-Throughput Sensor Data

```go
// IoT Architecture:
// Devices → MQTT Broker → Kafka (via bridge/connector) → Go Consumers → TimescaleDB

// Topic design for IoT:
// sensors.readings.{device_type}  — partitioned by device_id hash
// sensors.alerts                  — low volume, high priority
// sensors.metadata                — device registry events

// Partition count = max desired consumer parallelism
// For 100k devices, 50 partitions → 50 consumers per group

// Consumer for high-throughput sensor data
type SensorConsumer struct {
    reader  *kafka.Reader
    db      *pgxpool.Pool
    batch   []SensorReading
    metrics prometheus.Counter
}

func (c *SensorConsumer) Run(ctx context.Context) error {
    const batchSize = 500
    const flushInterval = 500 * time.Millisecond

    ticker := time.NewTicker(flushInterval)
    defer ticker.Stop()
```

```
    for {
        select {
        case <-ctx.Done(): return c.flushBatch(ctx)
        case <-ticker.C:
            if err := c.flushBatch(ctx); err != nil { return err }
        default:
            ctx2, cancel := context.WithTimeout(ctx, 100*time.Millisecond)
            msg, err := c.reader.FetchMessage(ctx2)
            cancel()
            if err != nil { continue }

            var r SensorReading
            json.Unmarshal(msg.Value, &r)
            c.batch = append(c.batch, r)

            if len(c.batch) >= batchSize {
                if err := c.flushBatch(ctx); err != nil { return err }
            }
        }
    }
}
```

8.4 Production Errors — Kafka

ERROR SECTION

ERROR: `kafka: offset out of range — OffsetOutOfRange`

ROOT CAUSE: Consumer's committed offset no longer exists (messages deleted by retention policy or topic was deleted/recreated).

FIX: Set auto.offset.reset = earliest (restart from beginning) or latest (skip ahead). Monitor consumer lag to catch up before retention.

ERROR: `kafka: rebalance in progress — RebalanceInProgress`

ROOT CAUSE: Consumer group rebalancing partitions (member joined/left/crashed). Consumers in the group receive this error during transition.

FIX: Handle gracefully: save state, finish in-flight processing, re-join group. Use static membership (group.instance.id) to reduce rebalances. Tune session.timeout.ms and heartbeat.interval.ms.

ERROR: `kafka: message too large — MessageTooLarge`

ROOT CAUSE: Message exceeds broker's message.max.bytes (default 1MB) or producer's max.request.size.

FIX: Increase broker: message.max.bytes and replica.fetch.max.bytes. Increase producer: max.request.size. OR compress message, OR split large messages, OR reference data in S3.

ERROR: `Consumer lag buildup — consumers can't keep up with producers`

ROOT CAUSE: Processing too slow relative to production rate. Causes: slow DB writes, single consumer per topic, expensive processing.

FIX: Increase partition count (allows more consumers). Add consumer instances. Batch DB writes (COPY FROM). Profile processing hot path. Use separate topics for high/low priority.

8.5 Interview Questions — Kafka

Q: What delivery guarantees does Kafka provide and how do you choose?

A: At-most-once: auto-commit before processing (fast, can lose messages). At-least-once: commit after processing (default choice, duplicates possible on retry — make processing idempotent). Exactly-once: Kafka transactions or idempotent producer + transactional consumer (complex, performance cost). For IoT: at-least-once with idempotency key in DB is practical.

Q: Why would you use key-based partitioning and what's the risk?

A: Key-based: messages with same key always go to same partition — guarantees ordering per device/user. Risk: hot partition if some keys have much higher volume (e.g., popular device or viral user). Solution: add entropy suffix for hot keys, or dedicated topics for high-volume producers.

Q: How do you monitor Kafka consumer health?

A: Consumer lag = (latest offset - committed offset) per partition. Alert on: lag > threshold (processing falling behind), lag growing over time (never catching up), consumer group member count dropping. Tools: kafka-consumer-groups.sh, Prometheus kafka-exporter, Burrow for lag analysis.

9. NATS

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

NATS is a lightweight, high-performance cloud-native messaging system. Core NATS provides pub/sub and request/reply with microsecond latency (no persistence). JetStream adds persistence, replay, and consumer management. Choose NATS over Kafka when you need simpler ops, lower latency, and request/reply patterns. Choose Kafka for durable ordered logs and large-scale stream processing.

9.1 Core NATS Patterns in Go

```go
import "github.com/nats-io/nats.go"

// Connect
nc, err := nats.Connect("nats://localhost:4222",
    nats.MaxReconnects(-1),       // reconnect forever
    nats.ReconnectWait(2*time.Second),
    nats.DisconnectErrHandler(func(nc *nats.Conn, err error) {
        log.Printf("disconnected: %v", err)
    }),
)
defer nc.Drain()  // gracefully flush pending messages

// Pub/Sub
nc.Publish("sensors.readings", data)  // fire and forget

sub, _ := nc.Subscribe("sensors.readings", func(msg *nats.Msg) {
    var reading SensorReading
    json.Unmarshal(msg.Data, &reading)
    processReading(reading)
})
defer sub.Unsubscribe()

// Queue Groups — load balancing across workers (like Kafka consumer groups)
nc.QueueSubscribe("sensors.readings", "processor-group", func(msg *nats.Msg) {
    // only ONE subscriber in the group receives each message
    process(msg.Data)
})

// Request/Reply — synchronous RPC over NATS
resp, err := nc.Request("user.lookup", []byte(`{"id":123}`), 5*time.Second)
if err != nil { return nil, fmt.Errorf("user lookup timeout: %w", err) }

// Service side (reply)
nc.Subscribe("user.lookup", func(msg *nats.Msg) {
    user := lookupUser(msg.Data)
    msg.Respond(user)  // sends to reply-to subject
})
```

9.2 JetStream — Persistence & Consumers

```go
// JetStream for durable messaging
js, err := nc.JetStream()

// Create stream (durable topic)
js.AddStream(&nats.StreamConfig{
    Name:      "SENSORS",
    Subjects:  []string{"sensors.>"},  // wildcard: sensors.temp, sensors.humidity, etc.
    Retention: nats.WorkQueuePolicy,   // delete on consumer ack
    MaxAge:    24 * time.Hour,
```

```
    Storage:   nats.FileStorage,       // persisted to disk
})

// Durable consumer (survives restart)
sub, _ := js.PullSubscribe("sensors.>", "sensor-processor",
    nats.Durable("sensor-processor"),
    nats.AckExplicit(),  // manual ack required
)

// Pull messages in batches
for {
    msgs, err := sub.Fetch(100, nats.MaxWait(1*time.Second))
    if err != nil && !errors.Is(err, nats.ErrTimeout) { return err }

    for _, msg := range msgs {
        if err := processMsg(msg); err != nil {
            msg.Nak()  // negative ack → redelivery
            continue
        }
        msg.Ack()  // positive ack → remove from stream
    }
}
```

9.3 Production Errors — NATS

ERROR SECTION

ERROR: `nats: slow consumer, messages dropped`

ROOT CAUSE: Subscriber processing messages slower than they arrive. NATS drops messages for slow consumers (unlike Kafka which applies backpressure).

FIX: Use queue groups with multiple workers. Or use JetStream (persists messages, subscriber can process at own pace). Increase pending limits: sub.SetPendingLimits(1000000, 256*1024*1024).

ERROR: `nats: maximum payload exceeded`

ROOT CAUSE: Message exceeds max_payload (default 1MB in server config).

FIX: Increase server max_payload in nats-server.conf. Or split large messages. Or store large data in S3 and send reference.

ERROR: `nats: connection closed / EOF`

ROOT CAUSE: Network interruption or server restart. Default client has reconnect logic but some messages may be lost.

FIX: Enable JetStream for guaranteed delivery. Use nats.MaxReconnects(-1) and nats.DrainTimeout(30s) for graceful shutdown.

9.4 Interview Questions — NATS

Q: When would you choose NATS over Kafka?

A: NATS: simpler ops (single binary, no ZooKeeper/KRaft), microsecond latency, request/reply pattern (no native equivalent in Kafka), lightweight service mesh messaging. JetStream adds enough durability for most use cases. Kafka: when you need guaranteed ordering, replay of historical events, large-scale stream processing with Kafka Streams/Flink, audit logs. For IoT real-time alerting NATS excels; for data pipeline with replay Kafka wins.

Q: Explain the difference between NATS core and JetStream.

A: Core NATS: fire-and-forget pub/sub. Messages not persisted — if subscriber is slow/down, messages are lost. Ideal for ephemeral real-time messaging. JetStream: at-rest persistence, consumer tracking (like Kafka consumer groups), replay from any offset, explicit ack/nak, max-age and size limits. Adds ~40% overhead vs core NATS but provides at-least-once delivery.

10. REDIS

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

Redis is an in-memory data structure store used as cache, message broker, and session store. Solves: slow DB reads (cache), distributed state (locks, rate limits, sessions), real-time data (leaderboards, pub/sub). Single-threaded (mostly) so no data races in Redis itself.

10.1 Data Structures with Go Examples

```go
import "github.com/redis/go-redis/v9"

rdb := redis.NewClient(&redis.Options{
    Addr:     "localhost:6379",
    PoolSize: 50,
    MinIdleConns: 10,
})

// STRING — simple cache
rdb.Set(ctx, "user:123", jsonBytes, 5*time.Minute)
val, err := rdb.Get(ctx, "user:123").Bytes()

// HASH — object fields (no serialization overhead for partial updates)
rdb.HSet(ctx, "user:123", "name", "Alice", "email", "a@b.com", "score", 100)
rdb.HIncrBy(ctx, "user:123", "score", 10)  // atomic increment
fields := rdb.HGetAll(ctx, "user:123").Val()

// LIST — queue/stack
rdb.LPush(ctx, "jobs", jobJSON)   // push to head
rdb.RPop(ctx, "jobs")             // pop from tail (FIFO queue)
rdb.BRPop(ctx, 5*time.Second, "jobs")  // blocking pop (worker pattern)

// SET — unique members, membership testing
rdb.SAdd(ctx, "online-users", userID)
rdb.SIsMember(ctx, "online-users", userID)

// SORTED SET — leaderboard
rdb.ZAdd(ctx, "leaderboard", redis.Z{Score: 1500.0, Member: "alice"})
rdb.ZIncrBy(ctx, "leaderboard", 50, "alice")  // atomic add
rdb.ZRevRank(ctx, "leaderboard", "alice")     // rank (0-indexed from top)
rdb.ZRevRangeWithScores(ctx, "leaderboard", 0, 9)  // top 10

// STREAM — append-only log (like mini-Kafka)
rdb.XAdd(ctx, &redis.XAddArgs{
    Stream: "sensor-events",
    MaxLen: 10000,  // cap stream size
    Values: map[string]any{"device_id": "d1", "temp": 23.5},
})
msgs, _ := rdb.XReadGroup(ctx, &redis.XReadGroupArgs{
    Group:    "processors",
    Consumer: "worker-1",
    Streams:  []string{"sensor-events", ">"},  // > = new messages
    Count:    100,
}).Result()
```

10.2 Distributed Locking — Redlock Algorithm

```go
// Simple SETNX lock (single Redis instance)
func acquireLock(ctx context.Context, rdb *redis.Client, key string, ttl time.Duration)
(bool, error) {
```

```go
    // SET key value NX PX ttl — atomic
    return rdb.SetNX(ctx, "lock:"+key, "locked", ttl).Result()
}

// Production Redlock with go-redsync (multiple Redis nodes)
import "github.com/go-redsync/redsync/v4"

rs := redsync.New(pool1, pool2, pool3)  // 3 independent Redis instances
mutex := rs.NewMutex("critical-section",
    redsync.WithExpiry(8*time.Second),
    redsync.WithRetryDelay(50*time.Millisecond),
    redsync.WithTries(3),
)

if err := mutex.LockContext(ctx); err != nil {
    return fmt.Errorf("acquire lock: %w", err)
}
defer mutex.Unlock()  // releases lock

// CRITICAL: always set TTL — no TTL = deadlock if process crashes
```

10.3 Caching Patterns

```go
// Cache-Aside (most common): app manages cache
func GetUser(ctx context.Context, id int64) (*User, error) {
    // 1. Try cache
    key := fmt.Sprintf("user:%d", id)
    cached, err := rdb.Get(ctx, key).Bytes()
    if err == nil {
        var u User
        json.Unmarshal(cached, &u)
        return &u, nil
    }

    // 2. Cache miss — load from DB
    user, err := db.GetUser(ctx, id)
    if errors.Is(err, ErrNotFound) {
        // Cache negative result to prevent cache penetration
        rdb.Set(ctx, key, "null", 30*time.Second)
        return nil, ErrNotFound
    }

    // 3. Populate cache
    data, _ := json.Marshal(user)
    rdb.Set(ctx, key, data, 10*time.Minute)
    return user, nil
}

// Eviction policies
// allkeys-lru:   evict least recently used from ALL keys (cache use case)
// volatile-lru:  evict LRU from keys WITH expiry set
// allkeys-lfu:   evict least frequently used (better for skewed access)
// noeviction:    return error when memory full (NOT for pure cache)
```

10.4 Production Errors — Redis

ERROR SECTION

ERROR: `MOVED 12345 192.168.1.2:6379 — Redis Cluster redirect`

ROOT CAUSE: Client sent command to wrong cluster node. Each key slot belongs to a specific node.

FIX: Use a cluster-aware client: go-redis automatically follows MOVED redirects. Never use pipeline across different key slots.

ERROR: `CROSSSLOT Keys in request don't hash to the same slot`

ROOT CAUSE: Multi-key command (MGET, pipeline) sent keys that belong to different cluster slots — not supported.

FIX: Use hash tags {} to force co-location: user:{123}:profile and user:{123}:settings share slot. Or use single-key operations.

ERROR: `OOM command not allowed when used memory > 'maxmemory'`

ROOT CAUSE: Redis hit memory limit and eviction policy is noeviction.

FIX: Change maxmemory-policy to allkeys-lru for cache use case. Increase maxmemory. Monitor memory with redis-cli info memory.

ERROR: `dial tcp: connection refused / pool timeout`

ROOT CAUSE: Redis connection pool exhausted. All PoolSize connections in use.

FIX: Increase PoolSize. Check for connection leaks (unreturned connections). Add timeouts: PoolTimeout, ReadTimeout, WriteTimeout.

10.5 Interview Questions — Redis

Q: Compare Redis Pub/Sub vs Streams vs Kafka.

A: Redis Pub/Sub: ephemeral, fire-and-forget, no persistence, subscriber must be online. Streams: persistent, consumer groups, replay from offset, ~Kafka-lite. Kafka: distributed, high-throughput, long retention, ecosystem (Kafka Streams, Connect). Use Redis Pub/Sub for real-time notifications (chat), Streams for lightweight event sourcing, Kafka for data pipeline.

Q: How do you prevent cache stampede?

A: Cache stampede: cache expires, N concurrent requests all miss and hit DB. Solutions: 1) Mutex/lock: first request acquires Redis lock, others wait or return stale. 2) Probabilistic early expiration: refresh before TTL ends stochastically. 3) Stale-while-revalidate: serve stale data, refresh in background. 4) Jitter on TTL to spread expiry times.

Q: When would you use Redis Cluster vs Sentinel?

A: Sentinel: HA for single primary. Monitors primary, promotes replica on failure. Suitable for <32GB datasets. Cluster: horizontal sharding across 16384 hash slots, automatically distributed across nodes. Suitable for large datasets or write-heavy workloads beyond single node. Cluster adds complexity (slot-aware clients, cross-slot limitations).

11. KUBERNETES

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

Kubernetes orchestrates containerized applications — scheduling, scaling, healing, networking, and rolling updates. Solves the problem of running microservices reliably across fleets of machines. Essential for production Go microservices at scale.

11.1 Production-Grade K8s Manifest for a Go Service

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-service
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1         # allow 1 extra pod during update
      maxUnavailable: 0   # never reduce below desired replicas
  template:
    metadata:
      labels:
        app: api-service
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "9090"
    spec:
      serviceAccountName: api-service
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        fsGroup: 1000
      containers:
      - name: api-service
        image: myregistry/api-service:v1.2.3  # ALWAYS use specific tag, never :latest
        ports:
        - containerPort: 8080
          name: http
        - containerPort: 9090
          name: metrics
        env:
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: api-secrets
              key: database-url
        - name: ENVIRONMENT
          value: "production"
        resources:
          requests:
            memory: "128Mi"
            cpu: "100m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        readinessProbe:
```

```
          httpGet:
            path: /readyz
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 10
          failureThreshold: 3
        livenessProbe:
          httpGet:
            path: /healthz
            port: 8080
          initialDelaySeconds: 15
          periodSeconds: 20
          failureThreshold: 3
        startupProbe:          # prevents liveness killing slow-start containers
          httpGet:
            path: /healthz
            port: 8080
          failureThreshold: 30  # 30 * 10s = 5 minutes max startup
          periodSeconds: 10
        lifecycle:
          preStop:
            exec:
              command: ["/bin/sh", "-c", "sleep 5"]  # allow LB to drain connections
```

11.2 HPA, Liveness vs Readiness vs Startup Probes

- Readiness: is the pod ready to serve traffic? Failed → removed from Service endpoints. Use for: DB connected, cache warmed, dependencies available.
- Liveness: is the pod alive? Failed → pod is killed and restarted. Use for: detecting deadlocks, leaked goroutines.
- Startup: is the pod finished starting? Overrides liveness until passes. Use for: slow initialization (ML model load, large cache warm).

```
// Health check handler in Go
func healthRouter(mux *http.ServeMux, db *pgxpool.Pool) {
    // Liveness: am I running? (simple)
    mux.HandleFunc("/healthz", func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
    })

    // Readiness: can I serve traffic? (check dependencies)
    mux.HandleFunc("/readyz", func(w http.ResponseWriter, r *http.Request) {
        ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
        defer cancel()

        if err := db.Ping(ctx); err != nil {
            http.Error(w, "database not ready: "+err.Error(),
http.StatusServiceUnavailable)
            return
        }
        w.WriteHeader(http.StatusOK)
    })
}

# HPA
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-service
```

```
  minReplicas: 3
  maxReplicas: 50
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Pods
    pods:
      metric:
        name: http_requests_per_second  # custom metric from Prometheus
      target:
        type: AverageValue
        averageValue: "1000"
```

11.3 Production Errors — Kubernetes

ERROR SECTION

ERROR: `CrashLoopBackOff — pod restarts repeatedly`

ROOT CAUSE: 5 common causes: 1) Application crash on startup (config error, panic). 2) Liveness probe too aggressive (kills healthy pod). 3) OOMKilled (memory limit too low). 4) Missing ConfigMap/Secret. 5) Command/ENTRYPOINT error.

FIX: Debug: kubectl logs <pod> --previous (logs before last restart). kubectl describe pod <pod> (exit code, reason). Exit 137=OOMKilled, Exit 1=app error.

```
# Debug CrashLoopBackOff
kubectl logs api-service-xxx --previous           # last crash logs
kubectl describe pod api-service-xxx              # events, exit code
kubectl get events --field-selector=reason=OOMKilling -n production

# Exit code meanings:
# 0 = success
# 1 = app error
# 137 = OOMKilled (SIGKILL, memory limit)
# 143 = graceful shutdown (SIGTERM)
# 139 = segfault
```

ERROR: `ImagePullBackOff — cannot pull image`

ROOT CAUSE: Image doesn't exist, wrong tag, or missing imagePullSecret for private registry.

FIX: Verify image exists: docker pull myregistry/image:tag. Check imagePullSecret is created and attached to ServiceAccount.

ERROR: `Pending pod — never scheduled`

ROOT CAUSE: Insufficient resources (CPU/memory requests exceed available nodes), or node selector/taint that no node satisfies.

FIX: kubectl describe pod <pending-pod>: look for 'Insufficient memory/cpu' or 'no nodes are available that match all of the predicates'. Add nodes or reduce resource requests.

ERROR: `OOMKilled — container killed (Exit 137)`

ROOT CAUSE: Process exceeded memory limits. In Go: goroutine leak, unbounded cache, not setting GOMEMLIMIT.

FIX: Set GOMEMLIMIT=80% of limit. Profile with pprof. Set realistic memory limits based on measured usage. Add memory alerts at 80% limit.

11.4 Interview Questions — Kubernetes

Q: What's the difference between requests and limits?

A: Requests: guaranteed resources, used by scheduler to place pod on a node. Limits: maximum allowed; CPU is throttled (not killed) when exceeded; memory is OOMKilled when exceeded. Best practice: set both, requests = typical load, limits = max safe for your SLA. Never set limits << requests. For Go: limit memory at GOMEMLIMIT + 20% headroom.

Q: How does a rolling update work and how do you ensure zero downtime?

A: Rolling update: gradually replaces old pods with new ones (controlled by maxSurge/maxUnavailable). Zero downtime requires: 1) readinessProbe — new pod only gets traffic when ready. 2) preStop sleep — allows existing requests to drain. 3) graceful shutdown — handle SIGTERM, complete in-flight requests. 4) minReadySeconds — pod must be ready N seconds before marking healthy.

Q: Explain RBAC in Kubernetes for a Go microservice.

A: ServiceAccount provides identity to pods. Role (namespace) or ClusterRole defines permissions. RoleBinding binds role to ServiceAccount. Go app uses ServiceAccount token automatically (mounted at /var/run/secrets/kubernetes.io/serviceaccount/token) for K8s API calls. Principle of least privilege: only grant permissions the service actually needs (e.g., only read ConfigMaps in own namespace).

12. PROMETHEUS & GRAFANA

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

Prometheus scrapes and stores time-series metrics. Grafana visualizes them. Together they provide observability for production services. Critical for detecting performance regressions, alerting on SLO breaches, and capacity planning.

12.1 Instrumenting a Go Service

```
import "github.com/prometheus/client_golang/prometheus"
import "github.com/prometheus/client_golang/prometheus/promauto"

// Counter: monotonically increasing (requests, errors)
var httpRequestsTotal = promauto.NewCounterVec(prometheus.CounterOpts{
    Name: "http_requests_total",
    Help: "Total number of HTTP requests",
}, []string{"method", "path", "status"})

// Gauge: can go up or down (goroutines, connections, queue depth)
var goroutinesGauge = promauto.NewGauge(prometheus.GaugeOpts{
    Name: "goroutines_count",
    Help: "Current number of goroutines",
})

// Histogram: request duration distribution (for p50, p99, p999)
var httpDuration = promauto.NewHistogramVec(prometheus.HistogramOpts{
    Name:    "http_request_duration_seconds",
    Help:    "HTTP request duration",
    Buckets: prometheus.DefBuckets, // .005,.01,.025,.05,.1,.25,.5,1,2.5,5,10
}, []string{"method", "path"})

// Middleware to instrument all HTTP handlers
func metricsMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        rw := &responseWriter{ResponseWriter: w, status: 200}
        next.ServeHTTP(rw, r)

        duration := time.Since(start).Seconds()
        path := chi.RouteContext(r.Context()).RoutePattern()  // use route pattern not
actual path

        httpRequestsTotal.WithLabelValues(r.Method, path, strconv.Itoa(rw.status)).Inc()
        httpDuration.WithLabelValues(r.Method, path).Observe(duration)
    })
}

// Golden Signals
// Latency: histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))
// Traffic:  rate(http_requests_total[5m])
// Errors:   rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m])
// Saturation: goroutines_count, cpu usage, memory
```

12.2 PromQL Queries

```
# Rate of requests per second over 5-minute window
rate(http_requests_total[5m])
```

```
# Error rate as percentage
rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m]) * 100

# 99th percentile latency
histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le,
path))

# Consumer lag (Kafka)
kafka_consumer_lag_sum > 10000

# Goroutine count growing (potential leak)
rate(goroutines_count[10m]) > 10

# Alertmanager rule
groups:
- name: api-alerts
  rules:
  - alert: HighErrorRate
    expr: rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m]) >
0.05
    for: 2m  # must be true for 2 minutes before firing
    labels:
      severity: critical
    annotations:
      summary: "High error rate: {{ $value | humanizePercentage }}"
      runbook_url: https://runbooks.internal/high-error-rate
```

12.3 Production Errors — Prometheus

ERROR SECTION

ERROR: `context deadline exceeded (scrape timeout)`

ROOT CAUSE: Prometheus scrape times out because /metrics endpoint is slow or the Go process is CPU-starved.

FIX: Increase scrape_timeout in prometheus.yml. Check if /metrics collection itself is slow (expensive computations). Use promauto to register collectors — they compute lazily on scrape.

ERROR: `Cardinality explosion — memory exhausted, too many time series`

ROOT CAUSE: Label with high cardinality (user_id, request_id, URL path with IDs) creates millions of unique time series.

FIX: NEVER use user IDs or UUIDs as label values. Use route pattern (/users/{id}) not actual path (/users/abc123). Audit cardinality: prometheus_tsdb_symbol_table_size_bytes growing rapidly.

ERROR: `Alert flapping — alert fires and resolves repeatedly`

ROOT CAUSE: Metric oscillates around threshold. Alert fires, metric dips below, resolves, rises, fires again.

FIX: Add 'for: 5m' to require sustained breach. Use smoothing: avg_over_time(metric[5m]). Add hysteresis: different thresholds for firing and resolving.

12.4 Interview Questions — Prometheus/Grafana

Q: What are the 4 Prometheus metric types and when do you use each?

A: Counter: only increases, resets on restart. For: request counts, error counts, bytes sent. rate() to get per-second rate. Gauge: can increase/decrease. For: current goroutines, memory usage, queue depth. Histogram: samples + buckets. For: request durations, payload sizes. Enables quantile calculation. Summary: similar to histogram but calculates quantiles client-side (not aggregatable). Prefer Histogram.

Q: Explain why you should use rate() instead of plain counter value.

A: Counters reset on restart (process restart, pod restart). rate() is resilient to resets and extrapolates per-second rate. Counter raw value grows forever — meaningless for alerting. rate(http_requests_total[5m]) gives you current req/s averaged over 5 minutes, adjusted for counter resets.

Q: How do you implement a Service Level Objective (SLO) in Prometheus?

A: Define SLIs: error rate = errors/total, latency = histogram_quantile(0.99,...). Set SLO: 99.9% requests succeed, p99 < 200ms. Alert on error budget burn rate: if burning 14.4x normal rate, page immediately (1h window). Tools: sloth.dev generates Prometheus rules from SLO definitions.

13. CI/CD PIPELINES

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

CI/CD automates testing, building, and deploying code changes. CI catches bugs early; CD ensures reliable, repeatable deployments. Critical for teams deploying multiple times per day.

13.1 GitHub Actions Pipeline for Go

```yaml
# .github/workflows/ci.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  GO_VERSION: "1.22"
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  test:
    name: Test & Lint
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_DB: testdb
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
        ports: ["5432:5432"]
        options: --health-cmd pg_isready --health-interval 10s

    steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-go@v5
      with:
        go-version: ${{ env.GO_VERSION }}
        cache: true  # caches $GOPATH/pkg/mod

    - name: golangci-lint
      uses: golangci/golangci-lint-action@v4
      with:
        version: latest
        args: --timeout=5m

    - name: go vet
      run: go vet ./...

    - name: Tests with race detector
      run: go test -race -coverprofile=coverage.out -covermode=atomic ./...
      env:
        DATABASE_URL: postgres://test:test@localhost:5432/testdb

    - name: Coverage threshold
      run: |
```

```
        COVERAGE=$(go tool cover -func coverage.out | grep total | awk '{print $3}' | tr
-d '%')
        echo "Coverage: $COVERAGE%"
        if (( $(echo "$COVERAGE < 80" | bc -l) )); then
          echo "Coverage $COVERAGE% below threshold 80%"
          exit 1
        fi

    - uses: codecov/codecov-action@v4

  build-push:
    name: Build & Push Image
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
    - uses: actions/checkout@v4

    - uses: docker/setup-buildx-action@v3

    - uses: docker/login-action@v3
      with:
        registry: ${{ env.REGISTRY }}
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}

    - name: Build and push
      uses: docker/build-push-action@v5
      with:
        context: .
        push: true
        tags: |
          ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
          ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:latest
        cache-from: type=gha   # GitHub Actions cache for Docker layers
        cache-to: type=gha,mode=max
        build-args: |
          VERSION=${{ github.sha }}

  deploy:
    name: Deploy to Production
    needs: build-push
    runs-on: ubuntu-latest
    environment: production  # requires manual approval

    steps:
    - uses: actions/checkout@v4

    - name: Deploy with kubectl
      run: |
        kubectl set image deployment/api-service \
          api-service=${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }} \
          --record
        kubectl rollout status deployment/api-service --timeout=300s
```

13.2 golangci-lint Configuration

```
# .golangci.yml
run:
  timeout: 5m
  tests: true
```

```
linters:
  enable:
    - errcheck      # check all errors are handled
    - gosimple      # simplifications
    - govet         # go vet checks
    - ineffassign   # detect ineffectual assignments
    - staticcheck   # advanced static analysis (SA series)
    - unused        # unused code
    - gofmt         # formatting
    - goimports     # import ordering
    - misspell      # spelling errors
    - revive        # replacement for golint
    - bodyclose     # http response body not closed
    - contextcheck  # context not passed to functions
    - gosec         # security checks
    - noctx         # http.NewRequest without context

issues:
  exclude-rules:
    - path: "_test.go"
      linters: [gosec, errcheck]
```

13.3 Production Errors — CI/CD

ERROR SECTION

ERROR: `Flaky tests — CI fails intermittently with different tests each time`

ROOT CAUSE: Race conditions in tests, time-dependent tests, port conflicts, shared state between tests.

FIX: Use t.Parallel() carefully. Mock time with fakeclock. Use random ports. Isolate state with table-driven tests and fresh DB transactions rolled back after each test.

ERROR: `Docker layer cache miss — every CI build downloads all dependencies`

ROOT CAUSE: Source code COPY before go mod download invalidates cache.

FIX: Always COPY go.mod go.sum first, RUN go mod download, THEN COPY . . This way dependencies layer only invalidates when go.mod changes.

ERROR: `Secret visible in CI logs — API key leaked`

ROOT CAUSE: Secret printed to stdout via fmt.Println, log.Printf, or curl -v showing Authorization header.

FIX: Use GitHub Secrets. Redact secrets in logs. Use --silent flag on curl. Audit logs with detect-secrets or trufflehog. Rotate leaked secrets immediately.

13.4 Interview Questions — CI/CD

Q: Explain the difference between blue-green and canary deployments.

A: Blue-green: run two identical environments (blue=live, green=new). Switch traffic all-at-once. Instant rollback by switching back. High resource cost (double infra). Canary: gradually shift traffic to new version (1% → 5% → 10% → 100%). Catch issues before full rollout. Monitor error rates per version. Rollback by setting canary weight to 0. Canary is safer for production; blue-green for faster rollback.

Q: What Go-specific checks do you add to CI?

A: go test -race (data race detector), go vet (correctness checks), golangci-lint (30+ linters), staticcheck (advanced analysis), go mod tidy (check go.mod is clean), coverage threshold (fail if below X%), go build with all build tags, benchmark regression check for performance-critical code.

14. NODE.JS / TYPESCRIPT (BONUS)

WHAT IT IS / WHAT PROBLEM IT SOLVES / WHEN TO USE IT

Node.js is a single-threaded event-loop runtime for JavaScript. TypeScript adds static typing. As a Go backend engineer, you'll encounter Node.js in BFF (Backend for Frontend) layers, build tooling, and legacy services.

14.1 Node.js Event Loop vs Go Goroutines

Aspect	Node.js	Go
Concurrency Model	Single-threaded event loop + worker threads	M:N goroutines on multiple OS threads
Blocking I/O	Non-blocking async I/O by design	Goroutine parks; M finds another G
CPU Work	Blocks event loop — offload to worker_threads	Just a goroutine — doesn't block others
Memory per unit	~1MB per worker thread	~2KB per goroutine
Typical Use	API gateway, BFF, real-time WebSocket	Backend services, high-throughput APIs

14.2 TypeScript for Reading BFF Code

```
// TypeScript interfaces — similar to Go interfaces but explicit
interface UserService {
    getUser(id: number): Promise<User>;
    createUser(req: CreateUserRequest): Promise<User>;
}

// Generics — similar to Go generics
function first<T>(arr: T[]): T | undefined {
    return arr[0];
}

// Utility types (unique to TypeScript)
type PartialUser = Partial<User>;            // all fields optional
type ReadonlyUser = Readonly<User>;           // all fields readonly
type UserPreview = Pick<User, 'id' | 'name'>;// only selected fields
type UserWithoutPwd = Omit<User, 'password'>;// exclude fields

// Async/await vs Go's goroutines
// Node.js
async function getUser(id: number): Promise<User> {
    const user = await db.query('SELECT * FROM users WHERE id = $1', [id]);
    if (!user) throw new NotFoundError(`user ${id} not found`);
    return user;
}

// Go equivalent
func (r *repo) GetUser(ctx context.Context, id int64) (*User, error) {
    var u User
    err := r.db.QueryRow(ctx, "SELECT * FROM users WHERE id = $1", id).Scan(&u)
    if errors.Is(err, pgx.ErrNoRows) { return nil, ErrNotFound }
    return &u, err
}
```

14.3 Interview Questions — Node.js/TypeScript

Q: How is Node.js event loop different from Go's concurrency model?

A: Node.js: single main thread with event loop. Callbacks, Promises, async/await are all cooperative — you yield at 'await'. CPU-bound work blocks everything. Go: N goroutines on M OS threads (GOMAXPROCS). I/O is non-blocking via goroutine parking. CPU work runs truly parallel. Go is better for CPU-bound work; Node.js is excellent for I/O-heavy work with simple code structure.

Q: When would a Go engineer need to write Node.js/TypeScript code?

A: 1) BFF (Backend for Frontend): thin layer aggregating multiple Go services for a specific UI. 2) Build tooling (webpack, esbuild configs). 3) Lambda functions for simple glue code. 4) CLI tools in a JS-native team. 5) Interfacing with npm packages (PDF generation, email templates with Handlebars).

DAY-OF-INTERVIEW CHEAT SHEET

One-liner definitions for every key term:

Goroutine: Lightweight user-space thread managed by Go runtime; starts at 2KB stack
G-M-P: Goroutine-Machine-Processor: Go's scheduling model mapping user to OS threads
Channel: Typed conduit for goroutine communication; unbuffered=synchronous, buffered=async
Context: Carries cancellation signals, deadlines, and request-scoped values through call stack
Escape Analysis: Compiler determines if var lives on stack (fast) or must go to heap (GC'd)
sync.Pool: Pool of reusable objects; reduces GC pressure for frequently allocated objects
CAP Theorem: Distributed system can guarantee only 2 of: Consistency, Availability, Partition tolerance
Circuit Breaker: Stops calling failing service after threshold; retries after recovery window
Saga Pattern: Distributed transaction via compensating actions; choreography or orchestration
CQRS: Command Query Responsibility Segregation: separate read and write models
Event Sourcing: Store events as source of truth; derive current state by replaying events
Consistent Hashing: Distributes keys to nodes on a ring; only K/N keys remapped when node added/removed
B-tree Index: Default PostgreSQL index; balanced tree for range and equality queries
GIN Index: Inverted index for JSONB, arrays, full-text search in PostgreSQL
MVCC: Multi-Version Concurrency Control; readers never block writers in PostgreSQL
N+1 Problem: 1 query for list + N queries for each item; fix with JOIN or batch fetch
pgxpool: PostgreSQL connection pool; MaxConns per app × instances should stay under max_connections
gRPC: Google RPC framework; Protocol Buffers over HTTP/2; 4 patterns: unary/server-stream/client-stream/bidi
Protocol Buffers: Binary serialization format; schema-first with .proto files; ~7x smaller than JSON
Interceptor: gRPC middleware; UnaryServerInterceptor for unary, StreamServerInterceptor for streaming
Kafka Partition: Unit of parallelism and ordering; one consumer per partition per consumer group
Consumer Lag: Gap between latest offset and committed offset; indicates processing falling behind
ISR: In-Sync Replicas: replicas fully caught up to leader; acks=all requires all ISR to acknowledge
Exactly-Once: Kafka semantic using transactions; or app-level idempotency key in DB
NATS JetStream: NATS persistence layer; adds consumer tracking, replay, explicit ack/nak
Redis Sorted Set: Key-score-member; O(log N) rank queries; perfect for leaderboards
Redlock: Distributed Redis lock across 3+ independent Redis instances; fault-tolerant
Cache Stampede: Multiple simultaneous cache misses hitting DB; fix: mutex, jitter, stale-while-revalidate
LRU Eviction: Least Recently Used; Redis allkeys-lru for pure cache workloads
Pod: Smallest K8s unit; one or more containers sharing network/storage
Readiness Probe: Is pod ready for traffic? Failure removes from Service endpoints
Liveness Probe: Is pod alive? Failure triggers pod restart
HPA: Horizontal Pod Autoscaler; scales replicas based on CPU/custom metrics
CrashLoopBackOff: Pod restarting repeatedly; debug with kubectl logs --previous
OOMKilled: Container exceeded memory limit; exit code 137; fix: profile + set GOMEMLIMIT
Prometheus Counter: Monotonically increasing metric; use rate() not raw value for alerting
Histogram: Samples + bucket counts; enables histogram_quantile() for p99 latency
Golden Signals: Latency, Traffic, Errors, Saturation — four metrics for any service
Cardinality: Number of unique time series; high cardinality (user IDs as labels) = OOM in Prometheus
pprof: Go profiling tool; CPU, heap, goroutine, mutex profiles accessible via HTTP endpoint
Race Detector: go test -race; instruments memory accesses at runtime; essential in CI
Multi-stage Build: Docker: separate build and runtime stages; Go binary in scratch/distroless = <20MB image
12-Factor App: Methodology for cloud-native apps: config in env vars, stateless processes, stdout logs
Outbox Pattern: Write event to DB table in same transaction; separate process publishes to Kafka
Distributed Tracing: Track requests across services via trace ID; OpenTelemetry + Jaeger/Zipkin
OpenTelemetry: Vendor-neutral observability standard; traces, metrics, logs with Go SDK
Token Bucket: Rate limiting: N tokens, refills at rate R; allows bursting

Sliding Window: Rate limiting: count requests in last N seconds; more accurate than fixed window

10 System Design Tips for IoT/Data Platforms

13. Start with scale numbers: 'How many devices? Events per second? Data retention?' — all design decisions flow from these
14. Partition by device_id for Kafka: ensures per-device ordering while maximizing parallelism
15. Batch DB writes: COPY FROM instead of INSERT for sensor data (100x throughput improvement)
16. Separate hot and cold paths: real-time alerts via NATS/Redis Streams; historical analysis via Kafka → TimescaleDB
17. Use BRIN indexes on timestamp columns in time-series tables (billions of rows, tiny index, great for range scans)
18. Design for backpressure: if consumer is slow, should producer block, drop, or buffer? Be explicit.
19. Idempotency everywhere: sensors retry on network failure; consumer must handle duplicate device readings (use device_id + sequence_num as unique key)
20. Monitor at every layer: device-side (missed heartbeats), Kafka lag, DB write latency, API p99
21. Plan for time zones and clock skew: always store UTC; handle devices with wrong clocks (reject readings > 5min in future)
22. Data lifecycle: raw events (7 days) → aggregated (1 year) → archived S3 (forever). Automate with TimescaleDB retention policies

5 Behavioral Questions — STAR Format Answers

Q1: Tell me about a time you improved system performance significantly.

STAR FRAMEWORK

Situation:

Our IoT ingestion service was writing sensor data row-by-row via INSERT, causing the PostgreSQL write path to become the bottleneck at 50k events/sec. CPU on the DB was 90% and we were dropping events.

Task:

Reduce DB write latency and increase throughput without changing the upstream Kafka pipeline.

Action:

Profiled with pprof — confirmed DB write was the bottleneck. Switched from individual INSERTs to batched COPY FROM using pgx CopyFrom with 500-record batches and 500ms flush interval. Added a secondary in-memory buffer with sync.Mutex to accumulate records before batch write. Tested with race detector. Deployed with feature flag.

Result:

Throughput increased from 50k to 800k events/sec. DB CPU dropped to 30%. Wrote runbook for the batching pattern, shared with team.

Q2: Describe a production incident you led and how you resolved it.

Situation:

3 AM alert: 40% of API requests returning 503. Kafka consumer lag at 500k. Service OOMKilled in Kubernetes.

Task:

Restore service within SLA (15 minutes), identify root cause.

Action:

kubectl describe pod → OOMKilled (Exit 137). Increased memory limit temporarily to restore service (5 min). Captured heap profile on staging reproduction. Found goroutine leak: HTTP client connections not closed when context cancelled. Each request leaked a goroutine holding 1MB connection buffer.

Result:

Fixed by adding defer resp.Body.Close() and setting http.Client timeouts. Added goleak test. Wrote post-mortem. Added memory usage alert at 80% limit.

Q3: Tell me about a time you disagreed with a technical decision and what happened.

Situation:

Team decided to use a shared PostgreSQL connection pool for 8 microservices, each up to 25 connections — total 200, near max_connections=200 default.

Task:

Raise concern without blocking the sprint.

Action:

Prepared data: showed that at peak load, we'd hit max_connections and services would fail to connect. Proposed PgBouncer in transaction pooling mode. Presented to tech lead with benchmarks showing PgBouncer at 1000 connections → 20 actual PostgreSQL connections with similar latency.

Result:

Team agreed to add PgBouncer. I led the implementation. We avoided the connection exhaustion issue that would have caused an incident at our next traffic spike.

Q4: How have you mentored junior engineers?

Situation:

Junior engineer on my team was submitting PRs with N+1 queries and no error handling, slowing down review cycles.

Task:

Improve code quality without demoralizing the engineer.

Action:

Scheduled weekly 1:1 technical deep-dives. Created a Go code review checklist covering: error wrapping, context propagation, SQL query patterns. Paired on their next feature, explaining the why behind each pattern. Set up golangci-lint to catch common issues automatically.

Result:

Within 2 months, their PRs required 50% fewer review cycles. They became the team's go-to person for PostgreSQL questions after mastering EXPLAIN ANALYZE.

Q5: Tell me about a complex distributed systems problem you solved.

Situation:

Two microservices had a dual-write problem: order-service wrote to PostgreSQL and published to Kafka, but if Kafka publish failed, orders were saved but events lost — silent inconsistency downstream.

Task:

Guarantee event delivery without distributed transactions.

Action:

Implemented Outbox Pattern: added events table to order-service DB. Wrote event to events table in the same transaction as the order. Created a separate relay process that polls events table and publishes to Kafka, marking events as published. Used exponential backoff with jitter for Kafka failures.

Result:

Achieved exactly-once semantics at the application level. Zero silent data loss incidents in 6 months post-implementation. Pattern adopted by 3 other services.